GenAI Team Lead Scenario Answers

Detailed Project Manager / Delivery Manager Interview Preparation

This document provides detailed answers for the complete scenario-based GenAI Team Lead question bank. Answers are framed around HealthGuard AI: a healthcare GenAI, RAG, voice assistant, patient-safety chatbot platform.

Use these answers to demonstrate end-to-end ownership: requirement discovery, architecture, RAG, LLM safety, voice UX, memory isolation, evaluation, performance, database design, security, DevOps, leadership, demo handling, and production readiness.

1. Project Understanding and Ownership

Team-lead lens: answer with business clarity, technical depth, safety awareness, measurable acceptance criteria, and delivery ownership.

Q1. You are assigned a GenAI healthcare chatbot project. What is the first thing you will do before writing code?

- I start by converting the idea into a controlled delivery plan before code. For HealthGuard AI, that means clarifying users, risk boundaries, data sources, safety constraints, MVP scope, and success metrics.
- My first step is discovery: confirm business goal, target users, clinical safety boundaries, data availability, compliance needs, success metrics, and MVP timeline.
- Implementation approach: I run discovery workshops, identify patient/doctor/admin/compliance workflows, define what the AI may and may not answer, map regulatory/privacy risks, and produce architecture plus backlog.
- Validation: I validate with an MVP scope document, acceptance criteria, prototype flows, risk register, and sign-off from product, clinical, security, and delivery stakeholders.
- Risk/trade-off: The main risk is building a chatbot that looks impressive but is unsafe, unmeasurable, or impossible to operate in production.

Q2. Client says, "We need a ChatGPT-like assistant for our company documents." How will you convert this vagu

- I start by converting the idea into a controlled delivery plan before code. For HealthGuard AI, that means clarifying users, risk boundaries, data sources, safety constraints, MVP scope, and success metrics.
- I convert vague input into a requirement checklist: user roles, top use cases, data sources, answer boundaries, latency, safety, integrations, reporting, and acceptance criteria.
- Implementation approach: I run discovery workshops, identify patient/doctor/admin/compliance workflows, define what the AI may and may not answer, map regulatory/privacy risks, and produce architecture plus backlog.
- Validation: I validate with an MVP scope document, acceptance criteria, prototype flows, risk register, and sign-off from product, clinical, security, and delivery stakeholders.
- Risk/trade-off: The main risk is building a chatbot that looks impressive but is unsafe, unmeasurable, or impossible to operate in production.

Q3. Product owner gives only one line: "Build an AI assistant for patients." What questions will you ask?

- I start by converting the idea into a controlled delivery plan before code. For HealthGuard AI, that means clarifying users, risk boundaries, data sources, safety constraints, MVP scope, and success metrics.
- I convert vague input into a requirement checklist: user roles, top use cases, data sources, answer boundaries, latency, safety, integrations, reporting, and acceptance criteria.
- Implementation approach: I run discovery workshops, identify patient/doctor/admin/compliance workflows, define what the AI may and may not answer, map regulatory/privacy risks, and produce architecture plus backlog.
- Validation: I validate with an MVP scope document, acceptance criteria, prototype flows, risk register, and sign-off from product, clinical, security, and delivery stakeholders.
- Risk/trade-off: The main risk is building a chatbot that looks impressive but is unsafe, unmeasurable, or impossible to operate in production.

Q4. How will you identify whether the project needs a simple chatbot, RAG, agents, fine-tuning, or a rule-based sy

- I start by converting the idea into a controlled delivery plan before code. For HealthGuard AI, that means clarifying users, risk boundaries, data sources, safety constraints, MVP scope, and success metrics.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: I run discovery workshops, identify patient/doctor/admin/compliance workflows, define what the AI may and may not answer, map regulatory/privacy risks, and produce architecture plus backlog.
- Validation: I validate with an MVP scope document, acceptance criteria, prototype flows, risk register, and sign-off from product, clinical, security, and delivery stakeholders.
- Risk/trade-off: The main risk is building a chatbot that looks impressive but is unsafe, unmeasurable, or impossible to operate in production.

Q5. Client wants the chatbot to answer everything. How will you explain the limitations of LLMs?

- I start by converting the idea into a controlled delivery plan before code. For HealthGuard AI, that means clarifying users, risk boundaries, data sources, safety constraints, MVP scope, and success metrics.
- As team lead, I split ownership clearly, review architecture and PRs, track risks, mentor developers, and translate technical trade-offs for stakeholders.
- Implementation approach: I run discovery workshops, identify patient/doctor/admin/compliance workflows, define what the AI may and may not answer, map regulatory/privacy risks, and produce architecture plus backlog.
- Validation: I validate with an MVP scope document, acceptance criteria, prototype flows, risk register, and sign-off from product, clinical, security, and delivery stakeholders.
- Risk/trade-off: The main risk is building a chatbot that looks impressive but is unsafe, unmeasurable, or impossible to operate in production.

Q6. How do you define project scope for a GenAI MVP?

- I start by converting the idea into a controlled delivery plan before code. For HealthGuard AI, that means clarifying users, risk boundaries, data sources, safety constraints, MVP scope, and success metrics.
- I run discovery workshops, identify patient/doctor/admin/compliance workflows, define what the AI may and may not answer, map regulatory/privacy risks, and produce architecture plus backlog.
- Implementation approach: I run discovery workshops, identify patient/doctor/admin/compliance workflows, define what the AI may and may not answer, map regulatory/privacy risks, and produce architecture plus backlog.
- Validation: I validate with an MVP scope document, acceptance criteria, prototype flows, risk register, and sign-off from product, clinical, security, and delivery stakeholders.
- Risk/trade-off: The main risk is building a chatbot that looks impressive but is unsafe, unmeasurable, or impossible to operate in production.

Q7. How do you decide what should be included in Phase 1, Phase 2, and Phase 3?

- I start by converting the idea into a controlled delivery plan before code. For HealthGuard AI, that means clarifying users, risk boundaries, data sources, safety constraints, MVP scope, and success metrics.
- I run discovery workshops, identify patient/doctor/admin/compliance workflows, define what the AI may and may not answer, map regulatory/privacy risks, and produce architecture plus backlog.
- Implementation approach: I run discovery workshops, identify patient/doctor/admin/compliance workflows, define what the AI may and may not answer, map regulatory/privacy risks, and produce architecture plus backlog.
- Validation: I validate with an MVP scope document, acceptance criteria, prototype flows, risk register, and sign-off from product, clinical, security, and delivery stakeholders.
- Risk/trade-off: The main risk is building a chatbot that looks impressive but is unsafe, unmeasurable, or impossible to operate in production.

Q8. What are the key risks in a GenAI project from day one?

- I start by converting the idea into a controlled delivery plan before code. For HealthGuard AI, that means clarifying users, risk boundaries, data sources, safety constraints, MVP scope, and success metrics.
- I run discovery workshops, identify patient/doctor/admin/compliance workflows, define what the AI may and may not answer, map regulatory/privacy risks, and produce architecture plus backlog.
- Implementation approach: I run discovery workshops, identify patient/doctor/admin/compliance workflows, define what the AI may and may not answer, map regulatory/privacy risks, and produce architecture plus backlog.
- Validation: I validate with an MVP scope document, acceptance criteria, prototype flows, risk register, and sign-off from product, clinical, security, and delivery stakeholders.
- Risk/trade-off: The main risk is building a chatbot that looks impressive but is unsafe, unmeasurable, or impossible to operate in production.

Q9. How do you estimate effort for a GenAI project when requirements are unclear?

- I start by converting the idea into a controlled delivery plan before code. For HealthGuard AI, that means clarifying users, risk boundaries, data sources, safety constraints, MVP scope, and success metrics.
- I run discovery workshops, identify patient/doctor/admin/compliance workflows, define what the AI may and may not answer, map regulatory/privacy risks, and produce architecture plus backlog.

- Implementation approach: I run discovery workshops, identify patient/doctor/admin/compliance workflows, define what the AI may and may not answer, map regulatory/privacy risks, and produce architecture plus backlog.
- Validation: I validate with an MVP scope document, acceptance criteria, prototype flows, risk register, and sign-off from product, clinical, security, and delivery stakeholders.
- Risk/trade-off: The main risk is building a chatbot that looks impressive but is unsafe, unmeasurable, or impossible to operate in production.

Q10. What documents will you prepare before starting development?

- I start by converting the idea into a controlled delivery plan before code. For HealthGuard AI, that means clarifying users, risk boundaries, data sources, safety constraints, MVP scope, and success metrics.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: I run discovery workshops, identify patient/doctor/admin/compliance workflows, define what the AI may and may not answer, map regulatory/privacy risks, and produce architecture plus backlog.
- Validation: I validate with an MVP scope document, acceptance criteria, prototype flows, risk register, and sign-off from product, clinical, security, and delivery stakeholders.
- Risk/trade-off: The main risk is building a chatbot that looks impressive but is unsafe, unmeasurable, or impossible to operate in production.

2. Requirement Analysis Scenarios

Team-lead lens: answer with business clarity, technical depth, safety awareness, measurable acceptance criteria, and delivery ownership.

Q11. User says: "The chatbot should give accurate answers." How do you convert this into measurable acceptan

- I convert vague GenAI requirements into measurable acceptance criteria. Terms like accurate, human-like, short, safe, or remembers must become testable behavior.
- I define intents, roles, inputs, outputs, response length, safety refusals, memory scope, upload rules, latency targets, evaluation datasets, and user journeys.
- Implementation approach: I define intents, roles, inputs, outputs, response length, safety refusals, memory scope, upload rules, latency targets, evaluation datasets, and user journeys.
- Validation: Acceptance criteria should include examples, negative tests, latency limits, citation requirements, red-flag handling, privacy rules, and demo scenarios.
- Risk/trade-off: If requirements stay subjective, teams argue about quality late in delivery and the client loses confidence.

Q12. Client says: "The bot should not hallucinate." What technical requirements will you define?

- I convert vague GenAI requirements into measurable acceptance criteria. Terms like accurate, human-like, short, safe, or remembers must become testable behavior.
- I combine RAG grounding, citation rules, answer refusal when context is missing, deterministic safety filters, prompt regression tests, and monitoring of unsupported claims.
- Implementation approach: I define intents, roles, inputs, outputs, response length, safety refusals, memory scope, upload rules, latency targets, evaluation datasets, and user journeys.
- Validation: Acceptance criteria should include examples, negative tests, latency limits, citation requirements, red-flag handling, privacy rules, and demo scenarios.
- Risk/trade-off: If requirements stay subjective, teams argue about quality late in delivery and the client loses confidence.

Q13. Client says: "The bot should remember user questions." What details will you clarify?

- I convert vague GenAI requirements into measurable acceptance criteria. Terms like accurate, human-like, short, safe, or remembers must become testable behavior.
- I require user_id and conversation_id scoping, saving the current user message before generation, latest-message retrieval, chronological prompt order, and tests for old-memory leakage.
- Implementation approach: I define intents, roles, inputs, outputs, response length, safety refusals, memory scope, upload rules, latency targets, evaluation datasets, and user journeys.
-

Validation: Acceptance criteria should include examples, negative tests, latency limits, citation requirements, red-flag handling, privacy rules, and demo scenarios.

- Risk/trade-off: If requirements stay subjective, teams argue about quality late in delivery and the client loses confidence.

Q14. Client says: "Voice input should work like human conversation." What requirements will you capture?

- I convert vague GenAI requirements into measurable acceptance criteria. Terms like accurate, human-like, short, safe, or remembers must become testable behavior.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: I define intents, roles, inputs, outputs, response length, safety refusals, memory scope, upload rules, latency targets, evaluation datasets, and user journeys.
- Validation: Acceptance criteria should include examples, negative tests, latency limits, citation requirements, red-flag handling, privacy rules, and demo scenarios.
- Risk/trade-off: If requirements stay subjective, teams argue about quality late in delivery and the client loses confidence.

Q15. Client says: "The bot should generate reports." What questions do you ask about report format, data source

- I convert vague GenAI requirements into measurable acceptance criteria. Terms like accurate, human-like, short, safe, or remembers must become testable behavior.
- I convert vague input into a requirement checklist: user roles, top use cases, data sources, answer boundaries, latency, safety, integrations, reporting, and acceptance criteria.
- Implementation approach: I define intents, roles, inputs, outputs, response length, safety refusals, memory scope, upload rules, latency targets, evaluation datasets, and user journeys.
- Validation: Acceptance criteria should include examples, negative tests, latency limits, citation requirements, red-flag handling, privacy rules, and demo scenarios.
- Risk/trade-off: If requirements stay subjective, teams argue about quality late in delivery and the client loses confidence.

Q16. Client says: "The chatbot answer should be short." How do you define response rules?

- I convert vague GenAI requirements into measurable acceptance criteria. Terms like accurate, human-like, short, safe, or remembers must become testable behavior.
- As team lead, I split ownership clearly, review architecture and PRs, track risks, mentor developers, and translate technical trade-offs for stakeholders.
- Implementation approach: I define intents, roles, inputs, outputs, response length, safety refusals, memory scope, upload rules, latency targets, evaluation datasets, and user journeys.
- Validation: Acceptance criteria should include examples, negative tests, latency limits, citation requirements, red-flag handling, privacy rules, and demo scenarios.
- Risk/trade-off: If requirements stay subjective, teams argue about quality late in delivery and the client loses confidence.

Q17. Client says: "It should work for doctors, patients, and admins." How will you design role-based requirement

- I convert vague GenAI requirements into measurable acceptance criteria. Terms like accurate, human-like, short, safe, or remembers must become testable behavior.
- As team lead, I split ownership clearly, review architecture and PRs, track risks, mentor developers, and translate technical trade-offs for stakeholders.
- Implementation approach: I define intents, roles, inputs, outputs, response length, safety refusals, memory scope, upload rules, latency targets, evaluation datasets, and user journeys.
- Validation: Acceptance criteria should include examples, negative tests, latency limits, citation requirements, red-flag handling, privacy rules, and demo scenarios.
- Risk/trade-off: If requirements stay subjective, teams argue about quality late in delivery and the client loses confidence.

Q18. Client says: "Upload documents and ask questions." What file-handling requirements do you collect?

- I convert vague GenAI requirements into measurable acceptance criteria. Terms like accurate, human-like, short, safe, or remembers must become testable behavior.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: I define intents, roles, inputs, outputs, response length, safety refusals, memory scope, upload rules, latency targets, evaluation datasets, and user journeys.
- Validation: Acceptance criteria should include examples, negative tests, latency limits, citation requirements, red-flag handling, privacy rules, and demo scenarios.
- Risk/trade-off: If requirements stay subjective, teams argue about quality late in delivery and the client loses confidence.

Q19. Client wants multilingual support. What questions do you ask before implementation?

- I convert vague GenAI requirements into measurable acceptance criteria. Terms like accurate, human-like, short, safe, or remembers must become testable behavior.
- I convert vague input into a requirement checklist: user roles, top use cases, data sources, answer boundaries, latency, safety, integrations, reporting, and acceptance criteria.
- Implementation approach: I define intents, roles, inputs, outputs, response length, safety refusals, memory scope, upload rules, latency targets, evaluation datasets, and user journeys.
- Validation: Acceptance criteria should include examples, negative tests, latency limits, citation requirements, red-flag handling, privacy rules, and demo scenarios.
- Risk/trade-off: If requirements stay subjective, teams argue about quality late in delivery and the client loses confidence.

Q20. Client wants the system in production in 4 weeks. How do you handle scope and expectations?

- I convert vague GenAI requirements into measurable acceptance criteria. Terms like accurate, human-like, short, safe, or remembers must become testable behavior.
- For delivery, I plan environments, Dockerization, secrets, migrations, observability, CI/CD, smoke tests, prompt versioning, and rollback.
- Implementation approach: I define intents, roles, inputs, outputs, response length, safety refusals, memory scope, upload rules, latency targets, evaluation datasets, and user journeys.
- Validation: Acceptance criteria should include examples, negative tests, latency limits, citation requirements, red-flag handling, privacy rules, and demo scenarios.
- Risk/trade-off: If requirements stay subjective, teams argue about quality late in delivery and the client loses confidence.

3. Architecture Design Scenarios

Team-lead lens: answer with business clarity, technical depth, safety awareness, measurable acceptance criteria, and delivery ownership.

Q21. Design a GenAI application architecture for document Q&A.

- I design the architecture around safety, data isolation, observability, and provider flexibility. The LLM is one component, not the whole system.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Use browser UI, FastAPI APIs, auth/RBAC, service layer, NLP intent analyzer, RAG pipeline, vector store, LLM abstraction, audit logs, workers, and monitoring.
- Validation: I review the architecture against scale, security, privacy, failure modes, cost, latency, and maintainability.
- Risk/trade-off: A weak architecture usually leaks data, mixes conversations, blocks API requests, or cannot survive LLM/provider failure.

Q22. Design a healthcare AI assistant architecture with safety guardrails.

- I design the architecture around safety, data isolation, observability, and provider flexibility. The LLM is one component, not the whole system.
- Use browser UI, FastAPI APIs, auth/RBAC, service layer, NLP intent analyzer, RAG pipeline, vector store, LLM abstraction, audit logs, workers, and monitoring.
- Implementation approach: Use browser UI, FastAPI APIs, auth/RBAC, service layer, NLP intent analyzer, RAG pipeline, vector store, LLM abstraction, audit logs, workers, and monitoring.
- Validation: I review the architecture against scale, security, privacy, failure modes, cost, latency, and maintainability.
- Risk/trade-off: A weak architecture usually leaks data, mixes conversations, blocks API requests, or cannot survive LLM/provider failure.

Q23. Design a chatbot that supports text, voice, and document upload.

- I design the architecture around safety, data isolation, observability, and provider flexibility. The LLM is one component, not the whole system.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Use browser UI, FastAPI APIs, auth/RBAC, service layer, NLP intent analyzer, RAG pipeline, vector store, LLM abstraction, audit logs, workers, and monitoring.
- Validation: I review the architecture against scale, security, privacy, failure modes, cost, latency, and maintainability.
- Risk/trade-off: A weak architecture usually leaks data, mixes conversations, blocks API requests, or cannot survive LLM/provider failure.

Q24. Design a RAG architecture for 1 million documents.

- I design the architecture around safety, data isolation, observability, and provider flexibility. The LLM is one component, not the whole system.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Use browser UI, FastAPI APIs, auth/RBAC, service layer, NLP intent analyzer, RAG pipeline, vector store, LLM abstraction, audit logs, workers, and monitoring.
- Validation: I review the architecture against scale, security, privacy, failure modes, cost, latency, and maintainability.
- Risk/trade-off: A weak architecture usually leaks data, mixes conversations, blocks API requests, or cannot survive LLM/provider failure.

Q25. Design a multi-user chatbot where each user has private memory.

- I design the architecture around safety, data isolation, observability, and provider flexibility. The LLM is one component, not the whole system.
- I require user_id and conversation_id scoping, saving the current user message before generation, latest-message retrieval, chronological prompt order, and tests for old-memory leakage.
- Implementation approach: Use browser UI, FastAPI APIs, auth/RBAC, service layer, NLP intent analyzer, RAG pipeline, vector store, LLM abstraction, audit logs, workers, and monitoring.
- Validation: I review the architecture against scale, security, privacy, failure modes, cost, latency, and maintainability.
- Risk/trade-off: A weak architecture usually leaks data, mixes conversations, blocks API requests, or cannot survive LLM/provider failure.

Q26. Design a system where doctors can review AI-generated reports.

- I design the architecture around safety, data isolation, observability, and provider flexibility. The LLM is one component, not the whole system.
- Use browser UI, FastAPI APIs, auth/RBAC, service layer, NLP intent analyzer, RAG pipeline, vector store, LLM abstraction, audit logs, workers, and monitoring.
- Implementation approach: Use browser UI, FastAPI APIs, auth/RBAC, service layer, NLP intent analyzer, RAG pipeline, vector store, LLM abstraction, audit logs, workers, and monitoring.
- Validation: I review the architecture against scale, security, privacy, failure modes, cost, latency, and maintainability.

- Risk/trade-off: A weak architecture usually leaks data, mixes conversations, blocks API requests, or cannot survive LLM/provider failure.

Q27. Design an admin workflow for uploading trusted knowledge documents.

- I design the architecture around safety, data isolation, observability, and provider flexibility. The LLM is one component, not the whole system.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Use browser UI, FastAPI APIs, auth/RBAC, service layer, NLP intent analyzer, RAG pipeline, vector store, LLM abstraction, audit logs, workers, and monitoring.
- Validation: I review the architecture against scale, security, privacy, failure modes, cost, latency, and maintainability.
- Risk/trade-off: A weak architecture usually leaks data, mixes conversations, blocks API requests, or cannot survive LLM/provider failure.

Q28. Design a fallback system when the LLM API fails.

- I design the architecture around safety, data isolation, observability, and provider flexibility. The LLM is one component, not the whole system.
- Use browser UI, FastAPI APIs, auth/RBAC, service layer, NLP intent analyzer, RAG pipeline, vector store, LLM abstraction, audit logs, workers, and monitoring.
- Implementation approach: Use browser UI, FastAPI APIs, auth/RBAC, service layer, NLP intent analyzer, RAG pipeline, vector store, LLM abstraction, audit logs, workers, and monitoring.
- Validation: I review the architecture against scale, security, privacy, failure modes, cost, latency, and maintainability.
- Risk/trade-off: A weak architecture usually leaks data, mixes conversations, blocks API requests, or cannot survive LLM/provider failure.

Q29. Design a system that supports OpenAI today and Gemini tomorrow.

- I design the architecture around safety, data isolation, observability, and provider flexibility. The LLM is one component, not the whole system.
- Use browser UI, FastAPI APIs, auth/RBAC, service layer, NLP intent analyzer, RAG pipeline, vector store, LLM abstraction, audit logs, workers, and monitoring.
- Implementation approach: Use browser UI, FastAPI APIs, auth/RBAC, service layer, NLP intent analyzer, RAG pipeline, vector store, LLM abstraction, audit logs, workers, and monitoring.
- Validation: I review the architecture against scale, security, privacy, failure modes, cost, latency, and maintainability.
- Risk/trade-off: A weak architecture usually leaks data, mixes conversations, blocks API requests, or cannot survive LLM/provider failure.

Q30. Design a scalable architecture for 10,000 concurrent chatbot users.

- I design the architecture around safety, data isolation, observability, and provider flexibility. The LLM is one component, not the whole system.
- Use browser UI, FastAPI APIs, auth/RBAC, service layer, NLP intent analyzer, RAG pipeline, vector store, LLM abstraction, audit logs, workers, and monitoring.
- Implementation approach: Use browser UI, FastAPI APIs, auth/RBAC, service layer, NLP intent analyzer, RAG pipeline, vector store, LLM abstraction, audit logs, workers, and monitoring.
- Validation: I review the architecture against scale, security, privacy, failure modes, cost, latency, and maintainability.
- Risk/trade-off: A weak architecture usually leaks data, mixes conversations, blocks API requests, or cannot survive LLM/provider failure.

4. RAG Pipeline Scenarios

Team-lead lens: answer with business clarity, technical depth, safety awareness, measurable acceptance criteria, and delivery ownership.

Q31. A user uploads a PDF. Explain the full flow until the chatbot answers from that PDF.

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.
- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q32. How do you extract text from PDF, Word, Excel, and scanned documents?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.
- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q33. How do you handle a PDF with tables, images, and handwritten notes?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.
- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q34. How do you decide chunk size and chunk overlap?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.
- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q35. What happens if chunks are too small?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.

- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q36. What happens if chunks are too large?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.
- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q37. How do you add metadata to chunks?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.
- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q38. How do you make sure one user cannot retrieve another user’s document chunks?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.
- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q39. How do you handle old documents and updated documents?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.
- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q40. How do you delete embeddings when a document is deleted?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.

- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.
- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q41. How do you retrieve the most relevant chunks?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.
- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q42. How do you improve retrieval quality if answers are wrong?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.
- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q43. How do you use reranking in RAG?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.
- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q44. How do you combine keyword search and vector search?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.
- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q45. How do you handle a question where no relevant document is found?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.
- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q46. How do you show citations or source references?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.
- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q47. How do you test whether RAG is working properly?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.
- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q48. What metrics do you use for RAG quality?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.
- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q49. How do you prevent the LLM from answering outside retrieved context?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.

- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

Q50. What is the difference between RAG and fine-tuning?

- For RAG, I focus on ingestion quality, metadata, retrieval quality, and answer grounding. Uploading a document is not enough; the retrieval pipeline must be measurable.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Validate files, extract text, clean, chunk, embed, store chunks with metadata, retrieve by semantic and keyword search, rerank, build prompt, cite sources, and filter by user_id.
- Validation: I test retrieval with known questions, expected chunks, citation accuracy, faithfulness, context precision/recall, and user isolation.
- Risk/trade-off: Poor chunking, missing metadata, or unfiltered retrieval creates wrong answers or cross-user data leakage.

5. LLM and Prompt Engineering Scenarios

Team-lead lens: answer with business clarity, technical depth, safety awareness, measurable acceptance criteria, and delivery ownership.

Q51. You need short answers only. How will you design the system prompt?

- Prompt engineering must be treated like versioned application logic. It needs requirements, tests, rollout, rollback, and monitoring.
- Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Implementation approach: Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Validation: Run prompt regression tests for style, safety, citations, hallucination, refusal, latency, and token cost before release.
- Risk/trade-off: Prompt-only safety is fragile. Backend rules must block medication dosage, diagnosis, unsafe medical advice, and prompt injection.

Q52. The chatbot gives long paragraphs. How do you fix it?

- Prompt engineering must be treated like versioned application logic. It needs requirements, tests, rollout, rollback, and monitoring.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Validation: Run prompt regression tests for style, safety, citations, hallucination, refusal, latency, and token cost before release.
- Risk/trade-off: Prompt-only safety is fragile. Backend rules must block medication dosage, diagnosis, unsafe medical advice, and prompt injection.

Q53. The chatbot gives unsafe medical advice. How do you fix it?

- Prompt engineering must be treated like versioned application logic. It needs requirements, tests, rollout, rollback, and monitoring.
- Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Implementation approach: Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Validation: Run prompt regression tests for style, safety, citations, hallucination, refusal, latency, and token cost before release.
- Risk/trade-off: Prompt-only safety is fragile. Backend rules must block medication dosage, diagnosis, unsafe medical advice, and prompt injection.

Q54. The chatbot repeats disclaimers every time. How do you improve UX?

- Prompt engineering must be treated like versioned application logic. It needs requirements, tests, rollout, rollback, and monitoring.
- Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Implementation approach: Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Validation: Run prompt regression tests for style, safety, citations, hallucination, refusal, latency, and token cost before release.
- Risk/trade-off: Prompt-only safety is fragile. Backend rules must block medication dosage, diagnosis, unsafe medical advice, and prompt injection.

Q55. The chatbot says "I don’t know" too often. How do you improve it without hallucination?

- Prompt engineering must be treated like versioned application logic. It needs requirements, tests, rollout, rollback, and monitoring.
- I combine RAG grounding, citation rules, answer refusal when context is missing, deterministic safety filters, prompt regression tests, and monitoring of unsupported claims.
- Implementation approach: Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Validation: Run prompt regression tests for style, safety, citations, hallucination, refusal, latency, and token cost before release.
- Risk/trade-off: Prompt-only safety is fragile. Backend rules must block medication dosage, diagnosis, unsafe medical advice, and prompt injection.

Q56. The chatbot hallucinates medicines. What prompt and backend rules will you add?

- Prompt engineering must be treated like versioned application logic. It needs requirements, tests, rollout, rollback, and monitoring.
- I combine RAG grounding, citation rules, answer refusal when context is missing, deterministic safety filters, prompt regression tests, and monitoring of unsupported claims.
- Implementation approach: Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Validation: Run prompt regression tests for style, safety, citations, hallucination, refusal, latency, and token cost before release.
- Risk/trade-off: Prompt-only safety is fragile. Backend rules must block medication dosage, diagnosis, unsafe medical advice, and prompt injection.

Q57. The chatbot answers without asking follow-up questions. How do you fix it?

- Prompt engineering must be treated like versioned application logic. It needs requirements, tests, rollout, rollback, and monitoring.
- Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Implementation approach: Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Validation: Run prompt regression tests for style, safety, citations, hallucination, refusal, latency, and token cost before release.
- Risk/trade-off: Prompt-only safety is fragile. Backend rules must block medication dosage, diagnosis, unsafe medical advice, and prompt injection.

Q58. How do you design a prompt for voice-based conversation?

- Prompt engineering must be treated like versioned application logic. It needs requirements, tests, rollout, rollback, and monitoring.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Validation: Run prompt regression tests for style, safety, citations, hallucination, refusal, latency, and token cost before release.

- Risk/trade-off: Prompt-only safety is fragile. Backend rules must block medication dosage, diagnosis, unsafe medical advice, and prompt injection.

Q59. How do you design prompts for JSON structured output?

- Prompt engineering must be treated like versioned application logic. It needs requirements, tests, rollout, rollback, and monitoring.
- Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Implementation approach: Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Validation: Run prompt regression tests for style, safety, citations, hallucination, refusal, latency, and token cost before release.
- Risk/trade-off: Prompt-only safety is fragile. Backend rules must block medication dosage, diagnosis, unsafe medical advice, and prompt injection.

Q60. How do you test prompt quality before production?

- Prompt engineering must be treated like versioned application logic. It needs requirements, tests, rollout, rollback, and monitoring.
- For delivery, I plan environments, Dockerization, secrets, migrations, observability, CI/CD, smoke tests, prompt versioning, and rollback.
- Implementation approach: Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Validation: Run prompt regression tests for style, safety, citations, hallucination, refusal, latency, and token cost before release.
- Risk/trade-off: Prompt-only safety is fragile. Backend rules must block medication dosage, diagnosis, unsafe medical advice, and prompt injection.

Q61. How do you version prompts?

- Prompt engineering must be treated like versioned application logic. It needs requirements, tests, rollout, rollback, and monitoring.
- Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Implementation approach: Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Validation: Run prompt regression tests for style, safety, citations, hallucination, refusal, latency, and token cost before release.
- Risk/trade-off: Prompt-only safety is fragile. Backend rules must block medication dosage, diagnosis, unsafe medical advice, and prompt injection.

Q62. How do you rollback a bad prompt release?

- Prompt engineering must be treated like versioned application logic. It needs requirements, tests, rollout, rollback, and monitoring.
- For delivery, I plan environments, Dockerization, secrets, migrations, observability, CI/CD, smoke tests, prompt versioning, and rollback.
- Implementation approach: Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Validation: Run prompt regression tests for style, safety, citations, hallucination, refusal, latency, and token cost before release.
- Risk/trade-off: Prompt-only safety is fragile. Backend rules must block medication dosage, diagnosis, unsafe medical advice, and prompt injection.

Q63. How do you compare two prompt versions?

- Prompt engineering must be treated like versioned application logic. It needs requirements, tests, rollout, rollback, and monitoring.
- Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Implementation approach: Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.

- Validation: Run prompt regression tests for style, safety, citations, hallucination, refusal, latency, and token cost before release.
- Risk/trade-off: Prompt-only safety is fragile. Backend rules must block medication dosage, diagnosis, unsafe medical advice, and prompt injection.

Q64. How do you reduce token usage in prompts?

- Prompt engineering must be treated like versioned application logic. It needs requirements, tests, rollout, rollback, and monitoring.
- Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Implementation approach: Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Validation: Run prompt regression tests for style, safety, citations, hallucination, refusal, latency, and token cost before release.
- Risk/trade-off: Prompt-only safety is fragile. Backend rules must block medication dosage, diagnosis, unsafe medical advice, and prompt injection.

Q65. How do you stop prompt injection attacks?

- Prompt engineering must be treated like versioned application logic. It needs requirements, tests, rollout, rollback, and monitoring.
- Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Implementation approach: Create system, RAG, safety, voice, concise-answer, and JSON prompts. Keep prompts short, explicit, and backed by backend rules.
- Validation: Run prompt regression tests for style, safety, citations, hallucination, refusal, latency, and token cost before release.
- Risk/trade-off: Prompt-only safety is fragile. Backend rules must block medication dosage, diagnosis, unsafe medical advice, and prompt injection.

6. Voice Assistant Scenarios

Team-lead lens: answer with business clarity, technical depth, safety awareness, measurable acceptance criteria, and delivery ownership.

Q66. Voice input stops while the user is still speaking. How do you fix it?

- Voice must behave like a conversation, not a file upload or assessment form. The core UX is listening, waiting through short pauses, finalizing transcript, then asking short follow-ups.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Validation: Test pauses, noisy speech, wrong transcript, unsupported browser, manual stop, mobile layout, and red-flag escalation.
- Risk/trade-off: If voice stops early or triggers report generation, users lose trust quickly.

Q67. User pauses for 2 seconds while speaking. Should voice stop? Why?

- Voice must behave like a conversation, not a file upload or assessment form. The core UX is listening, waiting through short pauses, finalizing transcript, then asking short follow-ups.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Validation: Test pauses, noisy speech, wrong transcript, unsupported browser, manual stop, mobile layout, and red-flag escalation.
- Risk/trade-off: If voice stops early or triggers report generation, users lose trust quickly.

Q68. How do you implement 5-second silence detection?

- Voice must behave like a conversation, not a file upload or assessment form. The core UX is listening, waiting through short pauses, finalizing transcript, then asking short follow-ups.
- Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Implementation approach: Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Validation: Test pauses, noisy speech, wrong transcript, unsupported browser, manual stop, mobile layout, and red-flag escalation.
- Risk/trade-off: If voice stops early or triggers report generation, users lose trust quickly.

Q69. How do you handle manual Stop button?

- Voice must behave like a conversation, not a file upload or assessment form. The core UX is listening, waiting through short pauses, finalizing transcript, then asking short follow-ups.
- Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Implementation approach: Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Validation: Test pauses, noisy speech, wrong transcript, unsupported browser, manual stop, mobile layout, and red-flag escalation.
- Risk/trade-off: If voice stops early or triggers report generation, users lose trust quickly.

Q70. How do you handle partial transcript and final transcript?

- Voice must behave like a conversation, not a file upload or assessment form. The core UX is listening, waiting through short pauses, finalizing transcript, then asking short follow-ups.
- Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Implementation approach: Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Validation: Test pauses, noisy speech, wrong transcript, unsupported browser, manual stop, mobile layout, and red-flag escalation.
- Risk/trade-off: If voice stops early or triggers report generation, users lose trust quickly.

Q71. Voice transcript is wrong. How do you recover?

- Voice must behave like a conversation, not a file upload or assessment form. The core UX is listening, waiting through short pauses, finalizing transcript, then asking short follow-ups.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Validation: Test pauses, noisy speech, wrong transcript, unsupported browser, manual stop, mobile layout, and red-flag escalation.
- Risk/trade-off: If voice stops early or triggers report generation, users lose trust quickly.

Q72. Voice query is unclear. How should chatbot respond?

- Voice must behave like a conversation, not a file upload or assessment form. The core UX is listening, waiting through short pauses, finalizing transcript, then asking short follow-ups.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Validation: Test pauses, noisy speech, wrong transcript, unsupported browser, manual stop, mobile layout, and red-flag escalation.
- Risk/trade-off: If voice stops early or triggers report generation, users lose trust quickly.

Q73. Voice asks: "Can I take medicine for fever?" What should bot say?

- Voice must behave like a conversation, not a file upload or assessment form. The core UX is listening, waiting through short pauses, finalizing transcript, then asking short follow-ups.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Validation: Test pauses, noisy speech, wrong transcript, unsupported browser, manual stop, mobile layout, and red-flag escalation.
- Risk/trade-off: If voice stops early or triggers report generation, users lose trust quickly.

Q74. Voice asks: "I have fever and rashes." What follow-up questions should bot ask?

- Voice must behave like a conversation, not a file upload or assessment form. The core UX is listening, waiting through short pauses, finalizing transcript, then asking short follow-ups.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Validation: Test pauses, noisy speech, wrong transcript, unsupported browser, manual stop, mobile layout, and red-flag escalation.
- Risk/trade-off: If voice stops early or triggers report generation, users lose trust quickly.

Q75. How do you make voice assistant conversational but safe?

- Voice must behave like a conversation, not a file upload or assessment form. The core UX is listening, waiting through short pauses, finalizing transcript, then asking short follow-ups.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Validation: Test pauses, noisy speech, wrong transcript, unsupported browser, manual stop, mobile layout, and red-flag escalation.
- Risk/trade-off: If voice stops early or triggers report generation, users lose trust quickly.

Q76. How do you avoid generating full reports from voice input?

- Voice must behave like a conversation, not a file upload or assessment form. The core UX is listening, waiting through short pauses, finalizing transcript, then asking short follow-ups.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Validation: Test pauses, noisy speech, wrong transcript, unsupported browser, manual stop, mobile layout, and red-flag escalation.
- Risk/trade-off: If voice stops early or triggers report generation, users lose trust quickly.

Q77. How do you make voice answers short?

- Voice must behave like a conversation, not a file upload or assessment form. The core UX is listening, waiting through short pauses, finalizing transcript, then asking short follow-ups.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Validation: Test pauses, noisy speech, wrong transcript, unsupported browser, manual stop, mobile layout, and red-flag escalation.
- Risk/trade-off: If voice stops early or triggers report generation, users lose trust quickly.

Q78. How do you store voice conversation history?

- Voice must behave like a conversation, not a file upload or assessment form. The core UX is listening, waiting through short pauses, finalizing transcript, then asking short follow-ups.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Validation: Test pauses, noisy speech, wrong transcript, unsupported browser, manual stop, mobile layout, and red-flag escalation.
- Risk/trade-off: If voice stops early or triggers report generation, users lose trust quickly.

Q79. How do you handle unsupported browsers for speech recognition?

- Voice must behave like a conversation, not a file upload or assessment form. The core UX is listening, waiting through short pauses, finalizing transcript, then asking short follow-ups.
- Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Implementation approach: Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Validation: Test pauses, noisy speech, wrong transcript, unsupported browser, manual stop, mobile layout, and red-flag escalation.
- Risk/trade-off: If voice stops early or triggers report generation, users lose trust quickly.

Q80. How do you test voice assistant UX?

- Voice must behave like a conversation, not a file upload or assessment form. The core UX is listening, waiting through short pauses, finalizing transcript, then asking short follow-ups.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Use continuous recognition, interim results, 5-second silence detection, manual stop, transcript correction, voice_mode routing, and short conversational responses.
- Validation: Test pauses, noisy speech, wrong transcript, unsupported browser, manual stop, mobile layout, and red-flag escalation.
- Risk/trade-off: If voice stops early or triggers report generation, users lose trust quickly.

7. Chat Memory Scenarios

Team-lead lens: answer with business clarity, technical depth, safety awareness, measurable acceptance criteria, and delivery ownership.

Q81. Chatbot remembers old questions instead of the latest one. How do you debug?

- Chat memory must be scoped and ordered. The latest active conversation should drive follow-up answers, while old memory is only supporting context.
- I require user_id and conversation_id scoping, saving the current user message before generation, latest-message retrieval, chronological prompt order, and tests for old-memory leakage.
- Implementation approach: Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Validation: Test latest-message memory, long conversations, old conversation isolation, cross-user isolation, and assistant failure after user-message save.
- Risk/trade-off: Bad memory logic causes the bot to answer the wrong question or leak data across users.

Q82. What database query will you check first?

- Chat memory must be scoped and ordered. The latest active conversation should drive follow-up answers, while old memory is only supporting context.
- Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Implementation approach: Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
-

Validation: Test latest-message memory, long conversations, old conversation isolation, cross-user isolation, and assistant failure after user-message save.

- Risk/trade-off: Bad memory logic causes the bot to answer the wrong question or leak data across users.

Q83. How do you fetch latest 10 messages correctly?

- Chat memory must be scoped and ordered. The latest active conversation should drive follow-up answers, while old memory is only supporting context.
- I require user_id and conversation_id scoping, saving the current user message before generation, latest-message retrieval, chronological prompt order, and tests for old-memory leakage.
- Implementation approach: Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Validation: Test latest-message memory, long conversations, old conversation isolation, cross-user isolation, and assistant failure after user-message save.
- Risk/trade-off: Bad memory logic causes the bot to answer the wrong question or leak data across users.

Q84. Why do we fetch latest messages in DESC order and reverse before sending to LLM?

- Chat memory must be scoped and ordered. The latest active conversation should drive follow-up answers, while old memory is only supporting context.
- I require user_id and conversation_id scoping, saving the current user message before generation, latest-message retrieval, chronological prompt order, and tests for old-memory leakage.
- Implementation approach: Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Validation: Test latest-message memory, long conversations, old conversation isolation, cross-user isolation, and assistant failure after user-message save.
- Risk/trade-off: Bad memory logic causes the bot to answer the wrong question or leak data across users.

Q85. How do you ensure memory is filtered by ‘user_id‘?

- Chat memory must be scoped and ordered. The latest active conversation should drive follow-up answers, while old memory is only supporting context.
- I require user_id and conversation_id scoping, saving the current user message before generation, latest-message retrieval, chronological prompt order, and tests for old-memory leakage.
- Implementation approach: Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Validation: Test latest-message memory, long conversations, old conversation isolation, cross-user isolation, and assistant failure after user-message save.
- Risk/trade-off: Bad memory logic causes the bot to answer the wrong question or leak data across users.

Q86. How do you ensure memory is filtered by ‘conversation_id‘?

- Chat memory must be scoped and ordered. The latest active conversation should drive follow-up answers, while old memory is only supporting context.
- I require user_id and conversation_id scoping, saving the current user message before generation, latest-message retrieval, chronological prompt order, and tests for old-memory leakage.
- Implementation approach: Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Validation: Test latest-message memory, long conversations, old conversation isolation, cross-user isolation, and assistant failure after user-message save.
- Risk/trade-off: Bad memory logic causes the bot to answer the wrong question or leak data across users.

Q87. How do you avoid mixing old conversations?

- Chat memory must be scoped and ordered. The latest active conversation should drive follow-up answers, while old memory is only supporting context.
- Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Implementation approach: Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Validation: Test latest-message memory, long conversations, old conversation isolation, cross-user isolation, and assistant failure after user-message save.

- Risk/trade-off: Bad memory logic causes the bot to answer the wrong question or leak data across users.

Q88. How do you handle multiple browser tabs with different conversations?

- Chat memory must be scoped and ordered. The latest active conversation should drive follow-up answers, while old memory is only supporting context.
- Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Implementation approach: Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Validation: Test latest-message memory, long conversations, old conversation isolation, cross-user isolation, and assistant failure after user-message save.
- Risk/trade-off: Bad memory logic causes the bot to answer the wrong question or leak data across users.

Q89. How do you store user and assistant messages?

- Chat memory must be scoped and ordered. The latest active conversation should drive follow-up answers, while old memory is only supporting context.
- Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Implementation approach: Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Validation: Test latest-message memory, long conversations, old conversation isolation, cross-user isolation, and assistant failure after user-message save.
- Risk/trade-off: Bad memory logic causes the bot to answer the wrong question or leak data across users.

Q90. When should the current user message be saved: before or after LLM call?

- Chat memory must be scoped and ordered. The latest active conversation should drive follow-up answers, while old memory is only supporting context.
- Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Implementation approach: Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Validation: Test latest-message memory, long conversations, old conversation isolation, cross-user isolation, and assistant failure after user-message save.
- Risk/trade-off: Bad memory logic causes the bot to answer the wrong question or leak data across users.

Q91. What happens if assistant response generation fails after user message is saved?

- Chat memory must be scoped and ordered. The latest active conversation should drive follow-up answers, while old memory is only supporting context.
- Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Implementation approach: Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Validation: Test latest-message memory, long conversations, old conversation isolation, cross-user isolation, and assistant failure after user-message save.
- Risk/trade-off: Bad memory logic causes the bot to answer the wrong question or leak data across users.

Q92. How do you retry safely?

- Chat memory must be scoped and ordered. The latest active conversation should drive follow-up answers, while old memory is only supporting context.
- Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Implementation approach: Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Validation: Test latest-message memory, long conversations, old conversation isolation, cross-user isolation, and assistant failure after user-message save.
- Risk/trade-off: Bad memory logic causes the bot to answer the wrong question or leak data across users.

Q93. How do you summarize long chat history?

- Chat memory must be scoped and ordered. The latest active conversation should drive follow-up answers, while old memory is only supporting context.
- Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Implementation approach: Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Validation: Test latest-message memory, long conversations, old conversation isolation, cross-user isolation, and assistant failure after user-message save.
- Risk/trade-off: Bad memory logic causes the bot to answer the wrong question or leak data across users.

Q94. How do you delete user memory?

- Chat memory must be scoped and ordered. The latest active conversation should drive follow-up answers, while old memory is only supporting context.
- I require user_id and conversation_id scoping, saving the current user message before generation, latest-message retrieval, chronological prompt order, and tests for old-memory leakage.
- Implementation approach: Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Validation: Test latest-message memory, long conversations, old conversation isolation, cross-user isolation, and assistant failure after user-message save.
- Risk/trade-off: Bad memory logic causes the bot to answer the wrong question or leak data across users.

Q95. How do you test chat memory isolation?

- Chat memory must be scoped and ordered. The latest active conversation should drive follow-up answers, while old memory is only supporting context.
- I require user_id and conversation_id scoping, saving the current user message before generation, latest-message retrieval, chronological prompt order, and tests for old-memory leakage.
- Implementation approach: Save user message first, fetch latest messages by user_id and conversation_id, order DESC, limit 5-10, reverse to chronological order, then build the prompt.
- Validation: Test latest-message memory, long conversations, old conversation isolation, cross-user isolation, and assistant failure after user-message save.
- Risk/trade-off: Bad memory logic causes the bot to answer the wrong question or leak data across users.

8. NLP / Intent Detection Scenarios

Team-lead lens: answer with business clarity, technical depth, safety awareness, measurable acceptance criteria, and delivery ownership.

Q96. User asks: "What kind of information is needed for better suggestions?" How should NLP classify this?

- The NLP layer turns raw user text into structured intent and entities before answer generation. This prevents generic or unsafe replies.
- Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Implementation approach: Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Validation: Use labeled examples, confusion matrix, entity-level accuracy, red-flag recall, spelling/voice-error tests, and production feedback loops.
- Risk/trade-off: If intent detection is weak, the bot either over-asks, under-asks, or gives unsafe answers.

Q97. User says: "I have fever." What missing fields should system detect?

- The NLP layer turns raw user text into structured intent and entities before answer generation. This prevents generic or unsafe replies.
- Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Implementation approach: Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
-

Validation: Use labeled examples, confusion matrix, entity-level accuracy, red-flag recall, spelling/voice-error tests, and production feedback loops.

- Risk/trade-off: If intent detection is weak, the bot either over-asks, under-asks, or gives unsafe answers.

Q98. User says: "I have fever for 3 days and rashes." What entities should be extracted?

- The NLP layer turns raw user text into structured intent and entities before answer generation. This prevents generic or unsafe replies.
- Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Implementation approach: Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Validation: Use labeled examples, confusion matrix, entity-level accuracy, red-flag recall, spelling/voice-error tests, and production feedback loops.
- Risk/trade-off: If intent detection is weak, the bot either over-asks, under-asks, or gives unsafe answers.

Q99. User says: "Can I take paracetamol?" What intent is this?

- The NLP layer turns raw user text into structured intent and entities before answer generation. This prevents generic or unsafe replies.
- Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Implementation approach: Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Validation: Use labeled examples, confusion matrix, entity-level accuracy, red-flag recall, spelling/voice-error tests, and production feedback loops.
- Risk/trade-off: If intent detection is weak, the bot either over-asks, under-asks, or gives unsafe answers.

Q100. User says: "My sugar is 180." What intent and follow-up are needed?

- The NLP layer turns raw user text into structured intent and entities before answer generation. This prevents generic or unsafe replies.
- Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Implementation approach: Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Validation: Use labeled examples, confusion matrix, entity-level accuracy, red-flag recall, spelling/voice-error tests, and production feedback loops.
- Risk/trade-off: If intent detection is weak, the bot either over-asks, under-asks, or gives unsafe answers.

Q101. User says: "I feel chest pain." What should the system do?

- The NLP layer turns raw user text into structured intent and entities before answer generation. This prevents generic or unsafe replies.
- Red flags should trigger calm urgent-care guidance immediately, with no long diagnostic discussion.
- Implementation approach: Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Validation: Use labeled examples, confusion matrix, entity-level accuracy, red-flag recall, spelling/voice-error tests, and production feedback loops.
- Risk/trade-off: If intent detection is weak, the bot either over-asks, under-asks, or gives unsafe answers.

Q102. How do you detect red-flag symptoms?

- The NLP layer turns raw user text into structured intent and entities before answer generation. This prevents generic or unsafe replies.
- Red flags should trigger calm urgent-care guidance immediately, with no long diagnostic discussion.
- Implementation approach: Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Validation: Use labeled examples, confusion matrix, entity-level accuracy, red-flag recall, spelling/voice-error tests, and production feedback loops.
- Risk/trade-off: If intent detection is weak, the bot either over-asks, under-asks, or gives unsafe answers.

Q103. How do you ask only one follow-up question at a time?

- The NLP layer turns raw user text into structured intent and entities before answer generation. This prevents generic or unsafe replies.
- Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Implementation approach: Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Validation: Use labeled examples, confusion matrix, entity-level accuracy, red-flag recall, spelling/voice-error tests, and production feedback loops.
- Risk/trade-off: If intent detection is weak, the bot either over-asks, under-asks, or gives unsafe answers.

Q104. How do you stop asking repeated questions?

- The NLP layer turns raw user text into structured intent and entities before answer generation. This prevents generic or unsafe replies.
- Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Implementation approach: Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Validation: Use labeled examples, confusion matrix, entity-level accuracy, red-flag recall, spelling/voice-error tests, and production feedback loops.
- Risk/trade-off: If intent detection is weak, the bot either over-asks, under-asks, or gives unsafe answers.

Q105. How do you summarize collected user answers?

- The NLP layer turns raw user text into structured intent and entities before answer generation. This prevents generic or unsafe replies.
- Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Implementation approach: Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Validation: Use labeled examples, confusion matrix, entity-level accuracy, red-flag recall, spelling/voice-error tests, and production feedback loops.
- Risk/trade-off: If intent detection is weak, the bot either over-asks, under-asks, or gives unsafe answers.

Q106. How do you combine NLP output with RAG context?

- The NLP layer turns raw user text into structured intent and entities before answer generation. This prevents generic or unsafe replies.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Validation: Use labeled examples, confusion matrix, entity-level accuracy, red-flag recall, spelling/voice-error tests, and production feedback loops.
- Risk/trade-off: If intent detection is weak, the bot either over-asks, under-asks, or gives unsafe answers.

Q107. Should NLP be rule-based, LLM-based, or hybrid?

- The NLP layer turns raw user text into structured intent and entities before answer generation. This prevents generic or unsafe replies.
- Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Implementation approach: Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Validation: Use labeled examples, confusion matrix, entity-level accuracy, red-flag recall, spelling/voice-error tests, and production feedback loops.
- Risk/trade-off: If intent detection is weak, the bot either over-asks, under-asks, or gives unsafe answers.

Q108. How do you validate NLP accuracy?

- The NLP layer turns raw user text into structured intent and entities before answer generation. This prevents generic or unsafe replies.
- Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Implementation approach: Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Validation: Use labeled examples, confusion matrix, entity-level accuracy, red-flag recall, spelling/voice-error tests, and production feedback loops.
- Risk/trade-off: If intent detection is weak, the bot either over-asks, under-asks, or gives unsafe answers.

Q109. How do you improve intent detection over time?

- The NLP layer turns raw user text into structured intent and entities before answer generation. This prevents generic or unsafe replies.
- Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Implementation approach: Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Validation: Use labeled examples, confusion matrix, entity-level accuracy, red-flag recall, spelling/voice-error tests, and production feedback loops.
- Risk/trade-off: If intent detection is weak, the bot either over-asks, under-asks, or gives unsafe answers.

Q110. How do you handle spelling mistakes or voice transcript errors?

- The NLP layer turns raw user text into structured intent and entities before answer generation. This prevents generic or unsafe replies.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Extract intent, symptoms, duration, severity, age, medicines, allergies, conditions, red flags, missing fields, and next action.
- Validation: Use labeled examples, confusion matrix, entity-level accuracy, red-flag recall, spelling/voice-error tests, and production feedback loops.
- Risk/trade-off: If intent detection is weak, the bot either over-asks, under-asks, or gives unsafe answers.

9. Safety and Guardrail Scenarios

Team-lead lens: answer with business clarity, technical depth, safety awareness, measurable acceptance criteria, and delivery ownership.

Q111. User asks: "Which medicine should I take?" What should bot do?

- Healthcare GenAI needs deterministic guardrails before LLM output. Safety is not optional and cannot be delegated only to prompts.
- Medication and dosage requests must be refused safely. The bot should ask for context and suggest clinician review, but it must not prescribe or adjust medicines.
- Implementation approach: Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Validation: Create safety acceptance tests for medication, dosage, red flags, diagnosis requests, report interpretation, and hallucinated medical advice.
- Risk/trade-off: Unsafe medical output is a product, legal, and patient-safety risk.

Q112. User asks: "Can I increase dosage?" What should bot do?

- Healthcare GenAI needs deterministic guardrails before LLM output. Safety is not optional and cannot be delegated only to prompts.
- Medication and dosage requests must be refused safely. The bot should ask for context and suggest clinician review, but it must not prescribe or adjust medicines.
- Implementation approach: Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
-

Validation: Create safety acceptance tests for medication, dosage, red flags, diagnosis requests, report interpretation, and hallucinated medical advice.

- Risk/trade-off: Unsafe medical output is a product, legal, and patient-safety risk.

Q113. User says: "I have chest pain and breathing difficulty." What should bot do?

- Healthcare GenAI needs deterministic guardrails before LLM output. Safety is not optional and cannot be delegated only to prompts.
- Red flags should trigger calm urgent-care guidance immediately, with no long diagnostic discussion.
- Implementation approach: Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Validation: Create safety acceptance tests for medication, dosage, red flags, diagnosis requests, report interpretation, and hallucinated medical advice.
- Risk/trade-off: Unsafe medical output is a product, legal, and patient-safety risk.

Q114. User uploads a report and asks: "Do I have cancer?" How should bot respond?

- Healthcare GenAI needs deterministic guardrails before LLM output. Safety is not optional and cannot be delegated only to prompts.
- Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Implementation approach: Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Validation: Create safety acceptance tests for medication, dosage, red flags, diagnosis requests, report interpretation, and hallucinated medical advice.
- Risk/trade-off: Unsafe medical output is a product, legal, and patient-safety risk.

Q115. User asks for diagnosis. How do you safely answer?

- Healthcare GenAI needs deterministic guardrails before LLM output. Safety is not optional and cannot be delegated only to prompts.
- Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Implementation approach: Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Validation: Create safety acceptance tests for medication, dosage, red flags, diagnosis requests, report interpretation, and hallucinated medical advice.
- Risk/trade-off: Unsafe medical output is a product, legal, and patient-safety risk.

Q116. How do you stop the system from giving dosage advice?

- Healthcare GenAI needs deterministic guardrails before LLM output. Safety is not optional and cannot be delegated only to prompts.
- Medication and dosage requests must be refused safely. The bot should ask for context and suggest clinician review, but it must not prescribe or adjust medicines.
- Implementation approach: Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Validation: Create safety acceptance tests for medication, dosage, red flags, diagnosis requests, report interpretation, and hallucinated medical advice.
- Risk/trade-off: Unsafe medical output is a product, legal, and patient-safety risk.

Q117. How do you handle emergency symptoms?

- Healthcare GenAI needs deterministic guardrails before LLM output. Safety is not optional and cannot be delegated only to prompts.
- Red flags should trigger calm urgent-care guidance immediately, with no long diagnostic discussion.
- Implementation approach: Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Validation: Create safety acceptance tests for medication, dosage, red flags, diagnosis requests, report interpretation, and hallucinated medical advice.
- Risk/trade-off: Unsafe medical output is a product, legal, and patient-safety risk.

Q118. How do you avoid scary language?

- Healthcare GenAI needs deterministic guardrails before LLM output. Safety is not optional and cannot be delegated only to prompts.
- Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Implementation approach: Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Validation: Create safety acceptance tests for medication, dosage, red flags, diagnosis requests, report interpretation, and hallucinated medical advice.
- Risk/trade-off: Unsafe medical output is a product, legal, and patient-safety risk.

Q119. How do you keep the response positive but medically safe?

- Healthcare GenAI needs deterministic guardrails before LLM output. Safety is not optional and cannot be delegated only to prompts.
- Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Implementation approach: Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Validation: Create safety acceptance tests for medication, dosage, red flags, diagnosis requests, report interpretation, and hallucinated medical advice.
- Risk/trade-off: Unsafe medical output is a product, legal, and patient-safety risk.

Q120. How do you define safety acceptance criteria?

- Healthcare GenAI needs deterministic guardrails before LLM output. Safety is not optional and cannot be delegated only to prompts.
- Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Implementation approach: Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Validation: Create safety acceptance tests for medication, dosage, red flags, diagnosis requests, report interpretation, and hallucinated medical advice.
- Risk/trade-off: Unsafe medical output is a product, legal, and patient-safety risk.

Q121. How do you test medical safety?

- Healthcare GenAI needs deterministic guardrails before LLM output. Safety is not optional and cannot be delegated only to prompts.
- Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Implementation approach: Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Validation: Create safety acceptance tests for medication, dosage, red flags, diagnosis requests, report interpretation, and hallucinated medical advice.
- Risk/trade-off: Unsafe medical output is a product, legal, and patient-safety risk.

Q122. How do you add human doctor review?

- Healthcare GenAI needs deterministic guardrails before LLM output. Safety is not optional and cannot be delegated only to prompts.
- Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Implementation approach: Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Validation: Create safety acceptance tests for medication, dosage, red flags, diagnosis requests, report interpretation, and hallucinated medical advice.
- Risk/trade-off: Unsafe medical output is a product, legal, and patient-safety risk.

Q123. How do you log unsafe attempts?

- Healthcare GenAI needs deterministic guardrails before LLM output. Safety is not optional and cannot be delegated only to prompts.
- Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Implementation approach: Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Validation: Create safety acceptance tests for medication, dosage, red flags, diagnosis requests, report interpretation, and hallucinated medical advice.
- Risk/trade-off: Unsafe medical output is a product, legal, and patient-safety risk.

Q124. How do you handle hallucinated medical advice?

- Healthcare GenAI needs deterministic guardrails before LLM output. Safety is not optional and cannot be delegated only to prompts.
- I combine RAG grounding, citation rules, answer refusal when context is missing, deterministic safety filters, prompt regression tests, and monitoring of unsupported claims.
- Implementation approach: Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Validation: Create safety acceptance tests for medication, dosage, red flags, diagnosis requests, report interpretation, and hallucinated medical advice.
- Risk/trade-off: Unsafe medical output is a product, legal, and patient-safety risk.

Q125. What is fail-closed behavior in healthcare GenAI?

- Healthcare GenAI needs deterministic guardrails before LLM output. Safety is not optional and cannot be delegated only to prompts.
- Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Implementation approach: Refuse diagnosis, prescription, dosage, and medication changes. Detect red flags, escalate emergencies, use calm language, log unsafe attempts, and add doctor review.
- Validation: Create safety acceptance tests for medication, dosage, red flags, diagnosis requests, report interpretation, and hallucinated medical advice.
- Risk/trade-off: Unsafe medical output is a product, legal, and patient-safety risk.

10. Evaluation Metrics Scenarios

Team-lead lens: answer with business clarity, technical depth, safety awareness, measurable acceptance criteria, and delivery ownership.

Q126. How do you prove the chatbot answer is accurate?

- Evaluation must cover answer correctness, grounding, safety, latency, and user experience. Manual testing alone is not enough.
- Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Implementation approach: Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Validation: Build gold datasets, run model A/B comparisons, monitor production feedback, and review failed cases weekly.
- Risk/trade-off: Without metrics, the team cannot prove improvement or detect regressions.

Q127. What metrics do you use for RAG?

- Evaluation must cover answer correctness, grounding, safety, latency, and user experience. Manual testing alone is not enough.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
-

Validation: Build gold datasets, run model A/B comparisons, monitor production feedback, and review failed cases weekly.

- Risk/trade-off: Without metrics, the team cannot prove improvement or detect regressions.

Q128. What metrics do you use for chatbot response quality?

- Evaluation must cover answer correctness, grounding, safety, latency, and user experience. Manual testing alone is not enough.
- Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Implementation approach: Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Validation: Build gold datasets, run model A/B comparisons, monitor production feedback, and review failed cases weekly.
- Risk/trade-off: Without metrics, the team cannot prove improvement or detect regressions.

Q129. What metrics do you use for safety?

- Evaluation must cover answer correctness, grounding, safety, latency, and user experience. Manual testing alone is not enough.
- Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Implementation approach: Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Validation: Build gold datasets, run model A/B comparisons, monitor production feedback, and review failed cases weekly.
- Risk/trade-off: Without metrics, the team cannot prove improvement or detect regressions.

Q130. What metrics do you use for voice assistant?

- Evaluation must cover answer correctness, grounding, safety, latency, and user experience. Manual testing alone is not enough.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Validation: Build gold datasets, run model A/B comparisons, monitor production feedback, and review failed cases weekly.
- Risk/trade-off: Without metrics, the team cannot prove improvement or detect regressions.

Q131. What is faithfulness?

- Evaluation must cover answer correctness, grounding, safety, latency, and user experience. Manual testing alone is not enough.
- Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Implementation approach: Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Validation: Build gold datasets, run model A/B comparisons, monitor production feedback, and review failed cases weekly.
- Risk/trade-off: Without metrics, the team cannot prove improvement or detect regressions.

Q132. What is answer relevance?

- Evaluation must cover answer correctness, grounding, safety, latency, and user experience. Manual testing alone is not enough.
- Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Implementation approach: Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Validation: Build gold datasets, run model A/B comparisons, monitor production feedback, and review failed cases weekly.

- Risk/trade-off: Without metrics, the team cannot prove improvement or detect regressions.

Q133. What is context precision?

- Evaluation must cover answer correctness, grounding, safety, latency, and user experience. Manual testing alone is not enough.
- Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Implementation approach: Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Validation: Build gold datasets, run model A/B comparisons, monitor production feedback, and review failed cases weekly.
- Risk/trade-off: Without metrics, the team cannot prove improvement or detect regressions.

Q134. What is context recall?

- Evaluation must cover answer correctness, grounding, safety, latency, and user experience. Manual testing alone is not enough.
- Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Implementation approach: Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Validation: Build gold datasets, run model A/B comparisons, monitor production feedback, and review failed cases weekly.
- Risk/trade-off: Without metrics, the team cannot prove improvement or detect regressions.

Q135. What is hallucination rate?

- Evaluation must cover answer correctness, grounding, safety, latency, and user experience. Manual testing alone is not enough.
- I combine RAG grounding, citation rules, answer refusal when context is missing, deterministic safety filters, prompt regression tests, and monitoring of unsupported claims.
- Implementation approach: Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Validation: Build gold datasets, run model A/B comparisons, monitor production feedback, and review failed cases weekly.
- Risk/trade-off: Without metrics, the team cannot prove improvement or detect regressions.

Q136. What is citation accuracy?

- Evaluation must cover answer correctness, grounding, safety, latency, and user experience. Manual testing alone is not enough.
- Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Implementation approach: Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Validation: Build gold datasets, run model A/B comparisons, monitor production feedback, and review failed cases weekly.
- Risk/trade-off: Without metrics, the team cannot prove improvement or detect regressions.

Q137. What is red-flag detection rate?

- Evaluation must cover answer correctness, grounding, safety, latency, and user experience. Manual testing alone is not enough.
- Red flags should trigger calm urgent-care guidance immediately, with no long diagnostic discussion.
- Implementation approach: Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Validation: Build gold datasets, run model A/B comparisons, monitor production feedback, and review failed cases weekly.
- Risk/trade-off: Without metrics, the team cannot prove improvement or detect regressions.

Q138. What is memory accuracy?

- Evaluation must cover answer correctness, grounding, safety, latency, and user experience. Manual testing alone is not enough.
- I require user_id and conversation_id scoping, saving the current user message before generation, latest-message retrieval, chronological prompt order, and tests for old-memory leakage.
- Implementation approach: Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Validation: Build gold datasets, run model A/B comparisons, monitor production feedback, and review failed cases weekly.
- Risk/trade-off: Without metrics, the team cannot prove improvement or detect regressions.

Q139. How do you create a test dataset for evaluation?

- Evaluation must cover answer correctness, grounding, safety, latency, and user experience. Manual testing alone is not enough.
- Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Implementation approach: Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Validation: Build gold datasets, run model A/B comparisons, monitor production feedback, and review failed cases weekly.
- Risk/trade-off: Without metrics, the team cannot prove improvement or detect regressions.

Q140. How do you compare model version A and B?

- Evaluation must cover answer correctness, grounding, safety, latency, and user experience. Manual testing alone is not enough.
- Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Implementation approach: Measure faithfulness, relevance, context precision/recall, citation accuracy, hallucination rate, red-flag detection, memory accuracy, and voice completion rate.
- Validation: Build gold datasets, run model A/B comparisons, monitor production feedback, and review failed cases weekly.
- Risk/trade-off: Without metrics, the team cannot prove improvement or detect regressions.

11. Performance and Scalability Scenarios

Team-lead lens: answer with business clarity, technical depth, safety awareness, measurable acceptance criteria, and delivery ownership.

Q141. Chatbot response is slow. How do you debug?

- I debug latency by breaking down time across frontend, API, retrieval, LLM, database, and workers. Then I optimize the bottleneck, not guesses.
- Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Implementation approach: Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Validation: Track P50/P95/P99 latency, throughput, queue length, provider latency, retrieval latency, and cost per request.
- Risk/trade-off: Large files, slow LLMs, and blocking API requests can make the product unusable under load.

Q142. RAG retrieval is slow. How do you optimize?

- I debug latency by breaking down time across frontend, API, retrieval, LLM, database, and workers. Then I optimize the bottleneck, not guesses.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
-

Validation: Track P50/P95/P99 latency, throughput, queue length, provider latency, retrieval latency, and cost per request.

- Risk/trade-off: Large files, slow LLMs, and blocking API requests can make the product unusable under load.

Q143. LLM API is taking 15 seconds. What do you do?

- I debug latency by breaking down time across frontend, API, retrieval, LLM, database, and workers. Then I optimize the bottleneck, not guesses.
- Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Implementation approach: Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Validation: Track P50/P95/P99 latency, throughput, queue length, provider latency, retrieval latency, and cost per request.
- Risk/trade-off: Large files, slow LLMs, and blocking API requests can make the product unusable under load.

Q144. How do you reduce token usage?

- I debug latency by breaking down time across frontend, API, retrieval, LLM, database, and workers. Then I optimize the bottleneck, not guesses.
- Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Implementation approach: Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Validation: Track P50/P95/P99 latency, throughput, queue length, provider latency, retrieval latency, and cost per request.
- Risk/trade-off: Large files, slow LLMs, and blocking API requests can make the product unusable under load.

Q145. How do you cache repeated questions?

- I debug latency by breaking down time across frontend, API, retrieval, LLM, database, and workers. Then I optimize the bottleneck, not guesses.
- Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Implementation approach: Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Validation: Track P50/P95/P99 latency, throughput, queue length, provider latency, retrieval latency, and cost per request.
- Risk/trade-off: Large files, slow LLMs, and blocking API requests can make the product unusable under load.

Q146. How do you process large files asynchronously?

- I debug latency by breaking down time across frontend, API, retrieval, LLM, database, and workers. Then I optimize the bottleneck, not guesses.
- Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Implementation approach: Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Validation: Track P50/P95/P99 latency, throughput, queue length, provider latency, retrieval latency, and cost per request.
- Risk/trade-off: Large files, slow LLMs, and blocking API requests can make the product unusable under load.

Q147. How do you handle 5000 Excel uploads?

- I debug latency by breaking down time across frontend, API, retrieval, LLM, database, and workers. Then I optimize the bottleneck, not guesses.
- Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Implementation approach: Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Validation: Track P50/P95/P99 latency, throughput, queue length, provider latency, retrieval latency, and cost per request.

- Risk/trade-off: Large files, slow LLMs, and blocking API requests can make the product unusable under load.

Q148. How do you process large PDFs?

- I debug latency by breaking down time across frontend, API, retrieval, LLM, database, and workers. Then I optimize the bottleneck, not guesses.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Validation: Track P50/P95/P99 latency, throughput, queue length, provider latency, retrieval latency, and cost per request.
- Risk/trade-off: Large files, slow LLMs, and blocking API requests can make the product unusable under load.

Q149. How do you design background workers?

- I debug latency by breaking down time across frontend, API, retrieval, LLM, database, and workers. Then I optimize the bottleneck, not guesses.
- Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Implementation approach: Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Validation: Track P50/P95/P99 latency, throughput, queue length, provider latency, retrieval latency, and cost per request.
- Risk/trade-off: Large files, slow LLMs, and blocking API requests can make the product unusable under load.

Q150. How do you use Redis/Celery?

- I debug latency by breaking down time across frontend, API, retrieval, LLM, database, and workers. Then I optimize the bottleneck, not guesses.
- Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Implementation approach: Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Validation: Track P50/P95/P99 latency, throughput, queue length, provider latency, retrieval latency, and cost per request.
- Risk/trade-off: Large files, slow LLMs, and blocking API requests can make the product unusable under load.

Q151. How do you avoid blocking FastAPI requests?

- I debug latency by breaking down time across frontend, API, retrieval, LLM, database, and workers. Then I optimize the bottleneck, not guesses.
- Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Implementation approach: Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Validation: Track P50/P95/P99 latency, throughput, queue length, provider latency, retrieval latency, and cost per request.
- Risk/trade-off: Large files, slow LLMs, and blocking API requests can make the product unusable under load.

Q152. When will you use async?

- I debug latency by breaking down time across frontend, API, retrieval, LLM, database, and workers. Then I optimize the bottleneck, not guesses.
- Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Implementation approach: Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Validation: Track P50/P95/P99 latency, throughput, queue length, provider latency, retrieval latency, and cost per request.
- Risk/trade-off: Large files, slow LLMs, and blocking API requests can make the product unusable under load.

Q153. When will you use threading?

- I debug latency by breaking down time across frontend, API, retrieval, LLM, database, and workers. Then I optimize the bottleneck, not guesses.
- Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Implementation approach: Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Validation: Track P50/P95/P99 latency, throughput, queue length, provider latency, retrieval latency, and cost per request.
- Risk/trade-off: Large files, slow LLMs, and blocking API requests can make the product unusable under load.

Q154. When will you use multiprocessing?

- I debug latency by breaking down time across frontend, API, retrieval, LLM, database, and workers. Then I optimize the bottleneck, not guesses.
- Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Implementation approach: Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Validation: Track P50/P95/P99 latency, throughput, queue length, provider latency, retrieval latency, and cost per request.
- Risk/trade-off: Large files, slow LLMs, and blocking API requests can make the product unusable under load.

Q155. How do you handle rate limits from LLM providers?

- I debug latency by breaking down time across frontend, API, retrieval, LLM, database, and workers. Then I optimize the bottleneck, not guesses.
- Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Implementation approach: Use async I/O, streaming, caching, token reduction, smaller classifiers, batch embeddings, background workers, rate-limit handling, and queue monitoring.
- Validation: Track P50/P95/P99 latency, throughput, queue length, provider latency, retrieval latency, and cost per request.
- Risk/trade-off: Large files, slow LLMs, and blocking API requests can make the product unusable under load.

12. Database and Data Design Scenarios

Team-lead lens: answer with business clarity, technical depth, safety awareness, measurable acceptance criteria, and delivery ownership.

Q156. What tables are needed for a GenAI healthcare chatbot?

- The data model must support ownership, auditability, versioning, retrieval, and deletion. For healthcare, isolation is as important as schema correctness.
- Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Implementation approach: Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Validation: Review foreign keys, indexes, user_id filters, migration strategy, backups, soft/hard delete policy, and restore drills.
- Risk/trade-off: Weak data design creates privacy issues, slow queries, and painful migrations.

Q157. How do you design chat conversations and messages?

- The data model must support ownership, auditability, versioning, retrieval, and deletion. For healthcare, isolation is as important as schema correctness.
- Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Implementation approach: Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
-

Validation: Review foreign keys, indexes, user_id filters, migration strategy, backups, soft/hard delete policy, and restore drills.

- Risk/trade-off: Weak data design creates privacy issues, slow queries, and painful migrations.

Q158. How do you store uploaded documents?

- The data model must support ownership, auditability, versioning, retrieval, and deletion. For healthcare, isolation is as important as schema correctness.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Validation: Review foreign keys, indexes, user_id filters, migration strategy, backups, soft/hard delete policy, and restore drills.
- Risk/trade-off: Weak data design creates privacy issues, slow queries, and painful migrations.

Q159. How do you store document chunks?

- The data model must support ownership, auditability, versioning, retrieval, and deletion. For healthcare, isolation is as important as schema correctness.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Validation: Review foreign keys, indexes, user_id filters, migration strategy, backups, soft/hard delete policy, and restore drills.
- Risk/trade-off: Weak data design creates privacy issues, slow queries, and painful migrations.

Q160. How do you store embeddings?

- The data model must support ownership, auditability, versioning, retrieval, and deletion. For healthcare, isolation is as important as schema correctness.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Validation: Review foreign keys, indexes, user_id filters, migration strategy, backups, soft/hard delete policy, and restore drills.
- Risk/trade-off: Weak data design creates privacy issues, slow queries, and painful migrations.

Q161. How do you store user roles?

- The data model must support ownership, auditability, versioning, retrieval, and deletion. For healthcare, isolation is as important as schema correctness.
- Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Implementation approach: Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Validation: Review foreign keys, indexes, user_id filters, migration strategy, backups, soft/hard delete policy, and restore drills.
- Risk/trade-off: Weak data design creates privacy issues, slow queries, and painful migrations.

Q162. How do you store doctor review comments?

- The data model must support ownership, auditability, versioning, retrieval, and deletion. For healthcare, isolation is as important as schema correctness.
- Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Implementation approach: Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Validation: Review foreign keys, indexes, user_id filters, migration strategy, backups, soft/hard delete policy, and restore drills.

- Risk/trade-off: Weak data design creates privacy issues, slow queries, and painful migrations.

Q163. How do you store audit logs?

- The data model must support ownership, auditability, versioning, retrieval, and deletion. For healthcare, isolation is as important as schema correctness.
- Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Implementation approach: Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Validation: Review foreign keys, indexes, user_id filters, migration strategy, backups, soft/hard delete policy, and restore drills.
- Risk/trade-off: Weak data design creates privacy issues, slow queries, and painful migrations.

Q164. How do you handle soft delete vs hard delete?

- The data model must support ownership, auditability, versioning, retrieval, and deletion. For healthcare, isolation is as important as schema correctness.
- Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Implementation approach: Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Validation: Review foreign keys, indexes, user_id filters, migration strategy, backups, soft/hard delete policy, and restore drills.
- Risk/trade-off: Weak data design creates privacy issues, slow queries, and painful migrations.

Q165. How do you handle document versioning?

- The data model must support ownership, auditability, versioning, retrieval, and deletion. For healthcare, isolation is as important as schema correctness.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Validation: Review foreign keys, indexes, user_id filters, migration strategy, backups, soft/hard delete policy, and restore drills.
- Risk/trade-off: Weak data design creates privacy issues, slow queries, and painful migrations.

Q166. How do you handle old reports?

- The data model must support ownership, auditability, versioning, retrieval, and deletion. For healthcare, isolation is as important as schema correctness.
- Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Implementation approach: Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Validation: Review foreign keys, indexes, user_id filters, migration strategy, backups, soft/hard delete policy, and restore drills.
- Risk/trade-off: Weak data design creates privacy issues, slow queries, and painful migrations.

Q167. How do you ensure patient data isolation?

- The data model must support ownership, auditability, versioning, retrieval, and deletion. For healthcare, isolation is as important as schema correctness.
- Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Implementation approach: Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Validation: Review foreign keys, indexes, user_id filters, migration strategy, backups, soft/hard delete policy, and restore drills.
- Risk/trade-off: Weak data design creates privacy issues, slow queries, and painful migrations.

Q168. How do you migrate SQLite to PostgreSQL?

- The data model must support ownership, auditability, versioning, retrieval, and deletion. For healthcare, isolation is as important as schema correctness.
- Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Implementation approach: Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Validation: Review foreign keys, indexes, user_id filters, migration strategy, backups, soft/hard delete policy, and restore drills.
- Risk/trade-off: Weak data design creates privacy issues, slow queries, and painful migrations.

Q169. How do you use pgvector or Qdrant?

- The data model must support ownership, auditability, versioning, retrieval, and deletion. For healthcare, isolation is as important as schema correctness.
- Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Implementation approach: Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Validation: Review foreign keys, indexes, user_id filters, migration strategy, backups, soft/hard delete policy, and restore drills.
- Risk/trade-off: Weak data design creates privacy issues, slow queries, and painful migrations.

Q170. How do you backup and restore production data?

- The data model must support ownership, auditability, versioning, retrieval, and deletion. For healthcare, isolation is as important as schema correctness.
- For delivery, I plan environments, Dockerization, secrets, migrations, observability, CI/CD, smoke tests, prompt versioning, and rollback.
- Implementation approach: Design users, sessions, profiles, assessments, reports, conversations, messages, documents, chunks, embeddings, knowledge, reviews, and audit logs.
- Validation: Review foreign keys, indexes, user_id filters, migration strategy, backups, soft/hard delete policy, and restore drills.
- Risk/trade-off: Weak data design creates privacy issues, slow queries, and painful migrations.

13. Security and Privacy Scenarios

Team-lead lens: answer with business clarity, technical depth, safety awareness, measurable acceptance criteria, and delivery ownership.

Q171. How do you protect patient data?

- Security starts with backend enforcement: authentication, RBAC, ownership filters, upload scanning, audit logs, secret management, and HTTPS.
- Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Implementation approach: Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Validation: Run role tests, cross-user isolation tests, upload security tests, audit checks, and dependency/security scans.
- Risk/trade-off: Frontend-only checks, leaked secrets, or unfiltered retrieval can expose patient data.

Q172. How do you prevent one user from seeing another user’s chat?

- Security starts with backend enforcement: authentication, RBAC, ownership filters, upload scanning, audit logs, secret management, and HTTPS.
- Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Implementation approach: Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
-

Validation: Run role tests, cross-user isolation tests, upload security tests, audit checks, and dependency/security scans.

- Risk/trade-off: Frontend-only checks, leaked secrets, or unfiltered retrieval can expose patient data.

Q173. How do you prevent one user from retrieving another user's document chunks?

- Security starts with backend enforcement: authentication, RBAC, ownership filters, upload scanning, audit logs, secret management, and HTTPS.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Validation: Run role tests, cross-user isolation tests, upload security tests, audit checks, and dependency/security scans.
- Risk/trade-off: Frontend-only checks, leaked secrets, or unfiltered retrieval can expose patient data.

Q174. How do you secure APIs?

- Security starts with backend enforcement: authentication, RBAC, ownership filters, upload scanning, audit logs, secret management, and HTTPS.
- Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Implementation approach: Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Validation: Run role tests, cross-user isolation tests, upload security tests, audit checks, and dependency/security scans.
- Risk/trade-off: Frontend-only checks, leaked secrets, or unfiltered retrieval can expose patient data.

Q175. Why is frontend role hiding not enough?

- Security starts with backend enforcement: authentication, RBAC, ownership filters, upload scanning, audit logs, secret management, and HTTPS.
- Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Implementation approach: Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Validation: Run role tests, cross-user isolation tests, upload security tests, audit checks, and dependency/security scans.
- Risk/trade-off: Frontend-only checks, leaked secrets, or unfiltered retrieval can expose patient data.

Q176. How do you implement RBAC?

- Security starts with backend enforcement: authentication, RBAC, ownership filters, upload scanning, audit logs, secret management, and HTTPS.
- Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Implementation approach: Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Validation: Run role tests, cross-user isolation tests, upload security tests, audit checks, and dependency/security scans.
- Risk/trade-off: Frontend-only checks, leaked secrets, or unfiltered retrieval can expose patient data.

Q177. How do you handle JWT/session expiry?

- Security starts with backend enforcement: authentication, RBAC, ownership filters, upload scanning, audit logs, secret management, and HTTPS.
- Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Implementation approach: Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Validation: Run role tests, cross-user isolation tests, upload security tests, audit checks, and dependency/security scans.

- Risk/trade-off: Frontend-only checks, leaked secrets, or unfiltered retrieval can expose patient data.

Q178. How do you secure environment variables?

- Security starts with backend enforcement: authentication, RBAC, ownership filters, upload scanning, audit logs, secret management, and HTTPS.
- Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Implementation approach: Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Validation: Run role tests, cross-user isolation tests, upload security tests, audit checks, and dependency/security scans.
- Risk/trade-off: Frontend-only checks, leaked secrets, or unfiltered retrieval can expose patient data.

Q179. Why should personal Gmail not be used in production?

- Security starts with backend enforcement: authentication, RBAC, ownership filters, upload scanning, audit logs, secret management, and HTTPS.
- For delivery, I plan environments, Dockerization, secrets, migrations, observability, CI/CD, smoke tests, prompt versioning, and rollback.
- Implementation approach: Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Validation: Run role tests, cross-user isolation tests, upload security tests, audit checks, and dependency/security scans.
- Risk/trade-off: Frontend-only checks, leaked secrets, or unfiltered retrieval can expose patient data.

Q180. How do you configure no-reply email?

- Security starts with backend enforcement: authentication, RBAC, ownership filters, upload scanning, audit logs, secret management, and HTTPS.
- Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Implementation approach: Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Validation: Run role tests, cross-user isolation tests, upload security tests, audit checks, and dependency/security scans.
- Risk/trade-off: Frontend-only checks, leaked secrets, or unfiltered retrieval can expose patient data.

Q181. How do you mask PHI in logs?

- Security starts with backend enforcement: authentication, RBAC, ownership filters, upload scanning, audit logs, secret management, and HTTPS.
- Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Implementation approach: Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Validation: Run role tests, cross-user isolation tests, upload security tests, audit checks, and dependency/security scans.
- Risk/trade-off: Frontend-only checks, leaked secrets, or unfiltered retrieval can expose patient data.

Q182. How do you handle audit logging?

- Security starts with backend enforcement: authentication, RBAC, ownership filters, upload scanning, audit logs, secret management, and HTTPS.
- Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Implementation approach: Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Validation: Run role tests, cross-user isolation tests, upload security tests, audit checks, and dependency/security scans.
- Risk/trade-off: Frontend-only checks, leaked secrets, or unfiltered retrieval can expose patient data.

Q183. How do you prevent prompt injection?

- Security starts with backend enforcement: authentication, RBAC, ownership filters, upload scanning, audit logs, secret management, and HTTPS.
- Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Implementation approach: Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Validation: Run role tests, cross-user isolation tests, upload security tests, audit checks, and dependency/security scans.
- Risk/trade-off: Frontend-only checks, leaked secrets, or unfiltered retrieval can expose patient data.

Q184. How do you scan uploaded files?

- Security starts with backend enforcement: authentication, RBAC, ownership filters, upload scanning, audit logs, secret management, and HTTPS.
- Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Implementation approach: Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Validation: Run role tests, cross-user isolation tests, upload security tests, audit checks, and dependency/security scans.
- Risk/trade-off: Frontend-only checks, leaked secrets, or unfiltered retrieval can expose patient data.

Q185. How do you handle consent for medical data?

- Security starts with backend enforcement: authentication, RBAC, ownership filters, upload scanning, audit logs, secret management, and HTTPS.
- Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Implementation approach: Protect APIs, secure env variables, isolate RAG chunks, mask PHI in logs, require consent, prevent prompt injection, and configure no-reply email properly.
- Validation: Run role tests, cross-user isolation tests, upload security tests, audit checks, and dependency/security scans.
- Risk/trade-off: Frontend-only checks, leaked secrets, or unfiltered retrieval can expose patient data.

14. DevOps and Deployment Scenarios

Team-lead lens: answer with business clarity, technical depth, safety awareness, measurable acceptance criteria, and delivery ownership.

Q186. How do you deploy this GenAI project?

- A production GenAI deployment needs more than running Uvicorn. It needs containers, managed data services, secrets, monitoring, CI/CD, rollback, and prompt release control.
- For delivery, I plan environments, Dockerization, secrets, migrations, observability, CI/CD, smoke tests, prompt versioning, and rollback.
- Implementation approach: Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Validation: Use staging, production, health checks, dashboards, logs, alerts, pipeline tests, and rollback rehearsals.
- Risk/trade-off: Demo-style deployment fails under real traffic, security review, or provider outages.

Q187. What services are required in production?

- A production GenAI deployment needs more than running Uvicorn. It needs containers, managed data services, secrets, monitoring, CI/CD, rollback, and prompt release control.
- For delivery, I plan environments, Dockerization, secrets, migrations, observability, CI/CD, smoke tests, prompt versioning, and rollback.
- Implementation approach: Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
-

Validation: Use staging, production, health checks, dashboards, logs, alerts, pipeline tests, and rollback rehearsals.

- Risk/trade-off: Demo-style deployment fails under real traffic, security review, or provider outages.

Q188. How do you containerize FastAPI?

- A production GenAI deployment needs more than running Uvicorn. It needs containers, managed data services, secrets, monitoring, CI/CD, rollback, and prompt release control.
- Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Implementation approach: Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Validation: Use staging, production, health checks, dashboards, logs, alerts, pipeline tests, and rollback rehearsals.
- Risk/trade-off: Demo-style deployment fails under real traffic, security review, or provider outages.

Q189. How do you deploy frontend and backend separately?

- A production GenAI deployment needs more than running Uvicorn. It needs containers, managed data services, secrets, monitoring, CI/CD, rollback, and prompt release control.
- For delivery, I plan environments, Dockerization, secrets, migrations, observability, CI/CD, smoke tests, prompt versioning, and rollback.
- Implementation approach: Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Validation: Use staging, production, health checks, dashboards, logs, alerts, pipeline tests, and rollback rehearsals.
- Risk/trade-off: Demo-style deployment fails under real traffic, security review, or provider outages.

Q190. How do you manage environment variables?

- A production GenAI deployment needs more than running Uvicorn. It needs containers, managed data services, secrets, monitoring, CI/CD, rollback, and prompt release control.
- Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Implementation approach: Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Validation: Use staging, production, health checks, dashboards, logs, alerts, pipeline tests, and rollback rehearsals.
- Risk/trade-off: Demo-style deployment fails under real traffic, security review, or provider outages.

Q191. How do you configure PostgreSQL and Redis?

- A production GenAI deployment needs more than running Uvicorn. It needs containers, managed data services, secrets, monitoring, CI/CD, rollback, and prompt release control.
- Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Implementation approach: Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Validation: Use staging, production, health checks, dashboards, logs, alerts, pipeline tests, and rollback rehearsals.
- Risk/trade-off: Demo-style deployment fails under real traffic, security review, or provider outages.

Q192. How do you configure HTTPS?

- A production GenAI deployment needs more than running Uvicorn. It needs containers, managed data services, secrets, monitoring, CI/CD, rollback, and prompt release control.
- Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Implementation approach: Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Validation: Use staging, production, health checks, dashboards, logs, alerts, pipeline tests, and rollback rehearsals.

- Risk/trade-off: Demo-style deployment fails under real traffic, security review, or provider outages.

Q193. How do you handle LLM API keys?

- A production GenAI deployment needs more than running Uvicorn. It needs containers, managed data services, secrets, monitoring, CI/CD, rollback, and prompt release control.
- Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Implementation approach: Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Validation: Use staging, production, health checks, dashboards, logs, alerts, pipeline tests, and rollback rehearsals.
- Risk/trade-off: Demo-style deployment fails under real traffic, security review, or provider outages.

Q194. How do you monitor production errors?

- A production GenAI deployment needs more than running Uvicorn. It needs containers, managed data services, secrets, monitoring, CI/CD, rollback, and prompt release control.
- For delivery, I plan environments, Dockerization, secrets, migrations, observability, CI/CD, smoke tests, prompt versioning, and rollback.
- Implementation approach: Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Validation: Use staging, production, health checks, dashboards, logs, alerts, pipeline tests, and rollback rehearsals.
- Risk/trade-off: Demo-style deployment fails under real traffic, security review, or provider outages.

Q195. How do you perform rollback?

- A production GenAI deployment needs more than running Uvicorn. It needs containers, managed data services, secrets, monitoring, CI/CD, rollback, and prompt release control.
- For delivery, I plan environments, Dockerization, secrets, migrations, observability, CI/CD, smoke tests, prompt versioning, and rollback.
- Implementation approach: Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Validation: Use staging, production, health checks, dashboards, logs, alerts, pipeline tests, and rollback rehearsals.
- Risk/trade-off: Demo-style deployment fails under real traffic, security review, or provider outages.

Q196. How do you create staging and production environments?

- A production GenAI deployment needs more than running Uvicorn. It needs containers, managed data services, secrets, monitoring, CI/CD, rollback, and prompt release control.
- For delivery, I plan environments, Dockerization, secrets, migrations, observability, CI/CD, smoke tests, prompt versioning, and rollback.
- Implementation approach: Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Validation: Use staging, production, health checks, dashboards, logs, alerts, pipeline tests, and rollback rehearsals.
- Risk/trade-off: Demo-style deployment fails under real traffic, security review, or provider outages.

Q197. How do you run database migrations?

- A production GenAI deployment needs more than running Uvicorn. It needs containers, managed data services, secrets, monitoring, CI/CD, rollback, and prompt release control.
- Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Implementation approach: Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Validation: Use staging, production, health checks, dashboards, logs, alerts, pipeline tests, and rollback rehearsals.
- Risk/trade-off: Demo-style deployment fails under real traffic, security review, or provider outages.

Q198. How do you set up CI/CD?

- A production GenAI deployment needs more than running Uvicorn. It needs containers, managed data services, secrets, monitoring, CI/CD, rollback, and prompt release control.
- For delivery, I plan environments, Dockerization, secrets, migrations, observability, CI/CD, smoke tests, prompt versioning, and rollback.
- Implementation approach: Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Validation: Use staging, production, health checks, dashboards, logs, alerts, pipeline tests, and rollback rehearsals.
- Risk/trade-off: Demo-style deployment fails under real traffic, security review, or provider outages.

Q199. What tests should run in pipeline?

- A production GenAI deployment needs more than running Uvicorn. It needs containers, managed data services, secrets, monitoring, CI/CD, rollback, and prompt release control.
- Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Implementation approach: Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Validation: Use staging, production, health checks, dashboards, logs, alerts, pipeline tests, and rollback rehearsals.
- Risk/trade-off: Demo-style deployment fails under real traffic, security review, or provider outages.

Q200. How do you release prompt changes safely?

- A production GenAI deployment needs more than running Uvicorn. It needs containers, managed data services, secrets, monitoring, CI/CD, rollback, and prompt release control.
- Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Implementation approach: Deploy frontend/backend separately, configure PostgreSQL, Redis, object storage, vector DB, HTTPS, API keys, migrations, smoke tests, and environment-specific config.
- Validation: Use staging, production, health checks, dashboards, logs, alerts, pipeline tests, and rollback rehearsals.
- Risk/trade-off: Demo-style deployment fails under real traffic, security review, or provider outages.

15. Team Lead and Delivery Scenarios

Team-lead lens: answer with business clarity, technical depth, safety awareness, measurable acceptance criteria, and delivery ownership.

Q201. You have 3 developers: frontend, backend, and AI engineer. How do you divide work?

- As a team lead, I translate product goals into workstreams, risks, reviews, and measurable delivery. I do not let GenAI remain a black box.
- As team lead, I split ownership clearly, review architecture and PRs, track risks, mentor developers, and translate technical trade-offs for stakeholders.
- Implementation approach: Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Validation: Track progress through milestones, demos, acceptance criteria, risk burn-down, bug trends, and evaluation metrics.
- Risk/trade-off: Without leadership discipline, teams over-focus on prompts and miss safety, data, testing, and production readiness.

Q202. Developer says RAG is working, but output is wrong. How do you review?

- As a team lead, I translate product goals into workstreams, risks, reviews, and measurable delivery. I do not let GenAI remain a black box.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.

- Validation: Track progress through milestones, demos, acceptance criteria, risk burn-down, bug trends, and evaluation metrics.
- Risk/trade-off: Without leadership discipline, teams over-focus on prompts and miss safety, data, testing, and production readiness.

Q203. Frontend developer says voice issue is browser limitation. How do you validate?

- As a team lead, I translate product goals into workstreams, risks, reviews, and measurable delivery. I do not let GenAI remain a black box.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Validation: Track progress through milestones, demos, acceptance criteria, risk burn-down, bug trends, and evaluation metrics.
- Risk/trade-off: Without leadership discipline, teams over-focus on prompts and miss safety, data, testing, and production readiness.

Q204. Backend developer did not filter by ‘user_id‘. How do you catch this in review?

- As a team lead, I translate product goals into workstreams, risks, reviews, and measurable delivery. I do not let GenAI remain a black box.
- As team lead, I split ownership clearly, review architecture and PRs, track risks, mentor developers, and translate technical trade-offs for stakeholders.
- Implementation approach: Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Validation: Track progress through milestones, demos, acceptance criteria, risk burn-down, bug trends, and evaluation metrics.
- Risk/trade-off: Without leadership discipline, teams over-focus on prompts and miss safety, data, testing, and production readiness.

Q205. AI engineer says hallucination cannot be avoided. How do you respond?

- As a team lead, I translate product goals into workstreams, risks, reviews, and measurable delivery. I do not let GenAI remain a black box.
- I combine RAG grounding, citation rules, answer refusal when context is missing, deterministic safety filters, prompt regression tests, and monitoring of unsupported claims.
- Implementation approach: Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Validation: Track progress through milestones, demos, acceptance criteria, risk burn-down, bug trends, and evaluation metrics.
- Risk/trade-off: Without leadership discipline, teams over-focus on prompts and miss safety, data, testing, and production readiness.

Q206. Client wants production in 2 weeks. Team says 6 weeks. How do you handle?

- As a team lead, I translate product goals into workstreams, risks, reviews, and measurable delivery. I do not let GenAI remain a black box.
- For delivery, I plan environments, Dockerization, secrets, migrations, observability, CI/CD, smoke tests, prompt versioning, and rollback.
- Implementation approach: Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Validation: Track progress through milestones, demos, acceptance criteria, risk burn-down, bug trends, and evaluation metrics.
- Risk/trade-off: Without leadership discipline, teams over-focus on prompts and miss safety, data, testing, and production readiness.

Q207. How do you track GenAI project progress?

- As a team lead, I translate product goals into workstreams, risks, reviews, and measurable delivery. I do not let GenAI remain a black box.
- Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Implementation approach: Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Validation: Track progress through milestones, demos, acceptance criteria, risk burn-down, bug trends, and evaluation metrics.
- Risk/trade-off: Without leadership discipline, teams over-focus on prompts and miss safety, data, testing, and production readiness.

Q208. What daily standup questions will you ask?

- As a team lead, I translate product goals into workstreams, risks, reviews, and measurable delivery. I do not let GenAI remain a black box.
- As team lead, I split ownership clearly, review architecture and PRs, track risks, mentor developers, and translate technical trade-offs for stakeholders.
- Implementation approach: Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Validation: Track progress through milestones, demos, acceptance criteria, risk burn-down, bug trends, and evaluation metrics.
- Risk/trade-off: Without leadership discipline, teams over-focus on prompts and miss safety, data, testing, and production readiness.

Q209. What technical review checklist will you use?

- As a team lead, I translate product goals into workstreams, risks, reviews, and measurable delivery. I do not let GenAI remain a black box.
- Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Implementation approach: Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Validation: Track progress through milestones, demos, acceptance criteria, risk burn-down, bug trends, and evaluation metrics.
- Risk/trade-off: Without leadership discipline, teams over-focus on prompts and miss safety, data, testing, and production readiness.

Q210. How do you handle blockers from LLM API failure?

- As a team lead, I translate product goals into workstreams, risks, reviews, and measurable delivery. I do not let GenAI remain a black box.
- Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Implementation approach: Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Validation: Track progress through milestones, demos, acceptance criteria, risk burn-down, bug trends, and evaluation metrics.
- Risk/trade-off: Without leadership discipline, teams over-focus on prompts and miss safety, data, testing, and production readiness.

Q211. How do you handle unclear requirements?

- As a team lead, I translate product goals into workstreams, risks, reviews, and measurable delivery. I do not let GenAI remain a black box.
- Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Implementation approach: Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Validation: Track progress through milestones, demos, acceptance criteria, risk burn-down, bug trends, and evaluation metrics.

- Risk/trade-off: Without leadership discipline, teams over-focus on prompts and miss safety, data, testing, and production readiness.

Q212. How do you mentor junior developers in GenAI?

- As a team lead, I translate product goals into workstreams, risks, reviews, and measurable delivery. I do not let GenAI remain a black box.
- As team lead, I split ownership clearly, review architecture and PRs, track risks, mentor developers, and translate technical trade-offs for stakeholders.
- Implementation approach: Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Validation: Track progress through milestones, demos, acceptance criteria, risk burn-down, bug trends, and evaluation metrics.
- Risk/trade-off: Without leadership discipline, teams over-focus on prompts and miss safety, data, testing, and production readiness.

Q213. How do you review pull requests for AI features?

- As a team lead, I translate product goals into workstreams, risks, reviews, and measurable delivery. I do not let GenAI remain a black box.
- Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Implementation approach: Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Validation: Track progress through milestones, demos, acceptance criteria, risk burn-down, bug trends, and evaluation metrics.
- Risk/trade-off: Without leadership discipline, teams over-focus on prompts and miss safety, data, testing, and production readiness.

Q214. How do you ensure coding standards?

- As a team lead, I translate product goals into workstreams, risks, reviews, and measurable delivery. I do not let GenAI remain a black box.
- Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Implementation approach: Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Validation: Track progress through milestones, demos, acceptance criteria, risk burn-down, bug trends, and evaluation metrics.
- Risk/trade-off: Without leadership discipline, teams over-focus on prompts and miss safety, data, testing, and production readiness.

Q215. How do you communicate technical risks to non-technical stakeholders?

- As a team lead, I translate product goals into workstreams, risks, reviews, and measurable delivery. I do not let GenAI remain a black box.
- Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Implementation approach: Split frontend, backend, AI/RAG, QA, DevOps tasks; run standups; review architecture; enforce tests; manage scope; and communicate risks early.
- Validation: Track progress through milestones, demos, acceptance criteria, risk burn-down, bug trends, and evaluation metrics.
- Risk/trade-off: Without leadership discipline, teams over-focus on prompts and miss safety, data, testing, and production readiness.

16. Client Demo Scenarios

Team-lead lens: answer with business clarity, technical depth, safety awareness, measurable acceptance criteria, and delivery ownership.

Q216. How will you demo HealthGuard AI to the client?

- A client demo should tell a safe product story: problem, workflow, value, guardrails, evidence, and limitations.
- As team lead, I split ownership clearly, review architecture and PRs, track risks, mentor developers, and translate technical trade-offs for stakeholders.
- Implementation approach: Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Validation: Prepare fallback data, offline examples, known demo prompts, LLM failure plan, release notes, and feedback capture.
- Risk/trade-off: Live GenAI demos can fail; a lead prepares deterministic paths and explains limitations honestly.

Q217. What scenarios will you show first?

- A client demo should tell a safe product story: problem, workflow, value, guardrails, evidence, and limitations.
- Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Implementation approach: Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Validation: Prepare fallback data, offline examples, known demo prompts, LLM failure plan, release notes, and feedback capture.
- Risk/trade-off: Live GenAI demos can fail; a lead prepares deterministic paths and explains limitations honestly.

Q218. How will you demo RAG?

- A client demo should tell a safe product story: problem, workflow, value, guardrails, evidence, and limitations.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Validation: Prepare fallback data, offline examples, known demo prompts, LLM failure plan, release notes, and feedback capture.
- Risk/trade-off: Live GenAI demos can fail; a lead prepares deterministic paths and explains limitations honestly.

Q219. How will you demo voice input?

- A client demo should tell a safe product story: problem, workflow, value, guardrails, evidence, and limitations.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Validation: Prepare fallback data, offline examples, known demo prompts, LLM failure plan, release notes, and feedback capture.
- Risk/trade-off: Live GenAI demos can fail; a lead prepares deterministic paths and explains limitations honestly.

Q220. How will you demo safety guardrails?

- A client demo should tell a safe product story: problem, workflow, value, guardrails, evidence, and limitations.
- Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Implementation approach: Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Validation: Prepare fallback data, offline examples, known demo prompts, LLM failure plan, release notes, and feedback capture.

- Risk/trade-off: Live GenAI demos can fail; a lead prepares deterministic paths and explains limitations honestly.

Q221. How will you demo memory?

- A client demo should tell a safe product story: problem, workflow, value, guardrails, evidence, and limitations.
- I require user_id and conversation_id scoping, saving the current user message before generation, latest-message retrieval, chronological prompt order, and tests for old-memory leakage.
- Implementation approach: Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Validation: Prepare fallback data, offline examples, known demo prompts, LLM failure plan, release notes, and feedback capture.
- Risk/trade-off: Live GenAI demos can fail; a lead prepares deterministic paths and explains limitations honestly.

Q222. How will you demo document upload?

- A client demo should tell a safe product story: problem, workflow, value, guardrails, evidence, and limitations.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Validation: Prepare fallback data, offline examples, known demo prompts, LLM failure plan, release notes, and feedback capture.
- Risk/trade-off: Live GenAI demos can fail; a lead prepares deterministic paths and explains limitations honestly.

Q223. How will you demo doctor review?

- A client demo should tell a safe product story: problem, workflow, value, guardrails, evidence, and limitations.
- Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Implementation approach: Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Validation: Prepare fallback data, offline examples, known demo prompts, LLM failure plan, release notes, and feedback capture.
- Risk/trade-off: Live GenAI demos can fail; a lead prepares deterministic paths and explains limitations honestly.

Q224. How will you demo admin knowledge upload?

- A client demo should tell a safe product story: problem, workflow, value, guardrails, evidence, and limitations.
- Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Implementation approach: Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Validation: Prepare fallback data, offline examples, known demo prompts, LLM failure plan, release notes, and feedback capture.
- Risk/trade-off: Live GenAI demos can fail; a lead prepares deterministic paths and explains limitations honestly.

Q225. How will you handle if LLM fails during demo?

- A client demo should tell a safe product story: problem, workflow, value, guardrails, evidence, and limitations.
- Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Implementation approach: Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.

- Validation: Prepare fallback data, offline examples, known demo prompts, LLM failure plan, release notes, and feedback capture.
- Risk/trade-off: Live GenAI demos can fail; a lead prepares deterministic paths and explains limitations honestly.

Q226. How will you explain limitations during demo?

- A client demo should tell a safe product story: problem, workflow, value, guardrails, evidence, and limitations.
- Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Implementation approach: Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Validation: Prepare fallback data, offline examples, known demo prompts, LLM failure plan, release notes, and feedback capture.
- Risk/trade-off: Live GenAI demos can fail; a lead prepares deterministic paths and explains limitations honestly.

Q227. How will you collect client feedback?

- A client demo should tell a safe product story: problem, workflow, value, guardrails, evidence, and limitations.
- As team lead, I split ownership clearly, review architecture and PRs, track risks, mentor developers, and translate technical trade-offs for stakeholders.
- Implementation approach: Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Validation: Prepare fallback data, offline examples, known demo prompts, LLM failure plan, release notes, and feedback capture.
- Risk/trade-off: Live GenAI demos can fail; a lead prepares deterministic paths and explains limitations honestly.

Q228. How will you convert feedback into backlog items?

- A client demo should tell a safe product story: problem, workflow, value, guardrails, evidence, and limitations.
- Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Implementation approach: Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Validation: Prepare fallback data, offline examples, known demo prompts, LLM failure plan, release notes, and feedback capture.
- Risk/trade-off: Live GenAI demos can fail; a lead prepares deterministic paths and explains limitations honestly.

Q229. How will you prioritize demo fixes?

- A client demo should tell a safe product story: problem, workflow, value, guardrails, evidence, and limitations.
- Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Implementation approach: Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Validation: Prepare fallback data, offline examples, known demo prompts, LLM failure plan, release notes, and feedback capture.
- Risk/trade-off: Live GenAI demos can fail; a lead prepares deterministic paths and explains limitations honestly.

Q230. How will you prepare a release note?

- A client demo should tell a safe product story: problem, workflow, value, guardrails, evidence, and limitations.
- Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Implementation approach: Demo login, assessment, chat, voice, document upload, RAG answer with citation, safety refusal, memory, doctor review, and admin knowledge.
- Validation: Prepare fallback data, offline examples, known demo prompts, LLM failure plan, release notes, and feedback capture.
- Risk/trade-off: Live GenAI demos can fail; a lead prepares deterministic paths and explains limitations honestly.

17. Deep Validation Questions

Team-lead lens: answer with business clarity, technical depth, safety awareness, measurable acceptance criteria, and delivery ownership.

Q231. Explain the full flow from user question to final answer.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q232. Explain the full flow from document upload to RAG answer.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q233. Explain the full flow from voice input to chatbot response.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q234. Explain how latest chat memory works.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- I require user_id and conversation_id scoping, saving the current user message before generation, latest-message retrieval, chronological prompt order, and tests for old-memory leakage.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
-

Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.

- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q235. Explain how you fixed old-memory bug.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- I require user_id and conversation_id scoping, saving the current user message before generation, latest-message retrieval, chronological prompt order, and tests for old-memory leakage.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q236. Explain how you prevent hallucination.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- I combine RAG grounding, citation rules, answer refusal when context is missing, deterministic safety filters, prompt regression tests, and monitoring of unsupported claims.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q237. Explain how you handle missing information.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q238. Explain how you detect medical red flags.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q239. Explain how you avoid medication recommendation.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q240. Explain how you handle LLM timeout.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q241. Explain how you handle no RAG results.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q242. Explain how chunking affects answer quality.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q243. Explain how vector similarity works.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q244. Explain how embeddings are generated.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q245. Explain how you evaluate faithfulness.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q246. Explain how you secure user-specific data.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q247. Explain how you test voice assistant.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q248. Explain how you reduce cost.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q249. Explain how you scale for many users.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q250. Explain what you would improve in next version.

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.

- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q1. How will you design the complete GenAI architecture?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q2. How will you avoid hallucination?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- I combine RAG grounding, citation rules, answer refusal when context is missing, deterministic safety filters, prompt regression tests, and monitoring of unsupported claims.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q3. How will you evaluate answer accuracy?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q4. How will you design RAG for user-specific documents?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q5. How will you prevent cross-user data leakage?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q6. How will you handle voice input stopping early?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- For voice, I require continuous listening, interim transcript handling, manual stop, 5-second silence detection, final-transcript submission, and lightweight chat mode.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q7. How will you implement chat memory correctly?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- I require user_id and conversation_id scoping, saving the current user message before generation, latest-message retrieval, chronological prompt order, and tests for old-memory leakage.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q8. How will you detect emergency symptoms?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Red flags should trigger calm urgent-care guidance immediately, with no long diagnostic discussion.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q9. How will you handle LLM failure?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q10. How will you estimate the project?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q11. How will you split tasks among team members?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- As team lead, I split ownership clearly, review architecture and PRs, track risks, mentor developers, and translate technical trade-offs for stakeholders.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q12. How will you review AI code?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q13. How will you test RAG quality?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- For document/RAG scenarios, I focus on extraction quality, chunking, metadata, vector retrieval, reranking, citations, and strict user isolation.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q14. How will you monitor production?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- For delivery, I plan environments, Dockerization, secrets, migrations, observability, CI/CD, smoke tests, prompt versioning, and rollback.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q15. How will you reduce LLM cost?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q16. How will you deploy safely?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- For delivery, I plan environments, Dockerization, secrets, migrations, observability, CI/CD, smoke tests, prompt versioning, and rollback.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q17. How will you handle client scope changes?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- As team lead, I split ownership clearly, review architecture and PRs, track risks, mentor developers, and translate technical trade-offs for stakeholders.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q18. How will you explain technical risk to management?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q19. How will you ensure healthcare safety?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

Q20. What would you improve in this project after MVP?

- For deep validation, I explain the real implementation flow, not generic AI theory. I connect user action to API, services, data, safety, retrieval, and monitoring.
- Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Implementation approach: Walk through request lifecycle, storage, NLP, RAG, LLM, safety, response persistence, and tests.
- Validation: Use concrete examples from HealthGuard AI: fever, rash, medicine refusal, document upload, voice follow-up, and memory bug fix.
- Risk/trade-off: If a candidate cannot explain edge cases, they probably did not own the implementation.

18. Strong Follow-Up Probe Answering Framework

- Why this approach? Tie the choice to safety, scalability, user trust, delivery speed, and maintainability.
- Alternatives considered: mention simpler chatbot, pure LLM, RAG, fine-tuning, agents, and rule-based systems where relevant.

- What can go wrong? Discuss hallucination, wrong retrieval, data leakage, latency, cost, browser issues, and unclear requirements.
- How to test? Use unit tests, integration tests, RAG evals, prompt regressions, red-flag tests, memory tests, and user acceptance scenarios.
- How to monitor? Track latency, errors, cost, retrieval quality, hallucination rate, feedback, queue length, and safety events.
- How to explain to client? Use workflow diagrams, demos, limitations, phased delivery, and risk register language.
- How to scale and secure? Use workers, caching, vector DB, RBAC, owner filters, encryption, audit logs, and HTTPS.

19. Scoring Rubric - How to Sound Like a Real Team Lead

```
Architecture    | Explains full flow, failure modes, and inte | Only says use LLM API
RAG             | Explains chunks, embeddings, metadata, retr | Only says upload docs
Safety          | Mentions no diagnosis, no dosage, red flags | Ignores medical risk
Memory          | Uses user_id + conversation_id and latest-m | Mixes old chats
Voice           | Handles silence, interim transcript, manual | Only says use mic
Evaluation      | Uses faithfulness, relevance, recall, safet | Manual testing only
Security        | RBAC, PHI, audit, isolation, prompt injecti | Frontend checks only
Delivery        | MVP, phases, risks, estimation, stakeholder | No planning approach
Leadership      | Splits tasks, reviews, mentors, manages blo | Only individual coding
Production      | Monitoring, CI/CD, rollback, secrets, healt | Demo-only thinking
```

Best interview line: A strong GenAI Team Lead should not only know LLM, RAG, and prompts. They should know how to deliver a safe, scalable, secure, tested, monitored, production-ready AI system with a team.